



Gruppo Telegram

# Tutorato Algoritmi e laboratorio

## A.A. 2023/2024

### Outline:

- Alberi red-black (Teoria)
- Alberi red-black (Esercizi)
- Alberi red-black (Esercizi da esame)

## Cosa sono e a che servono?

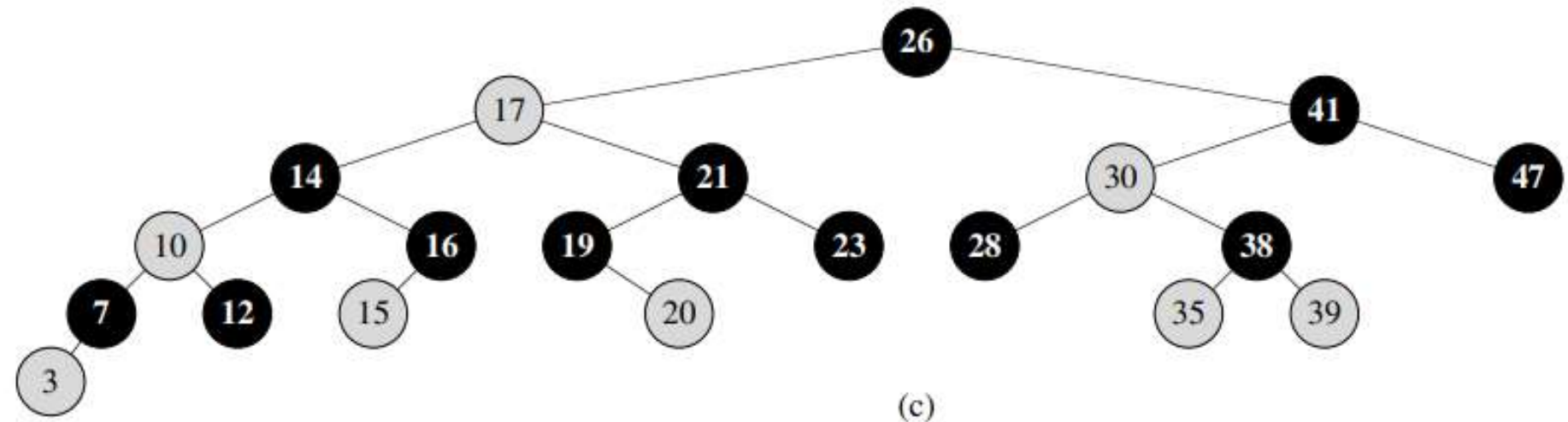
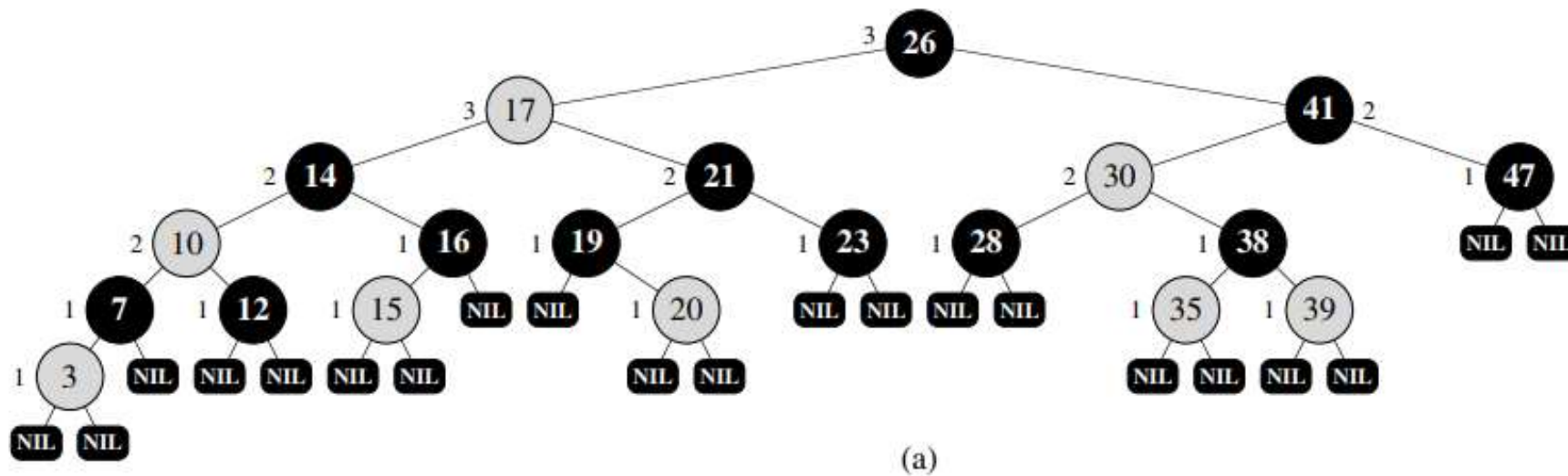
- Sappiamo come un albero binario di ricerca con un'altezza  $h$  possa eseguire diverse operazioni fondamentali su insiemi dinamici, quali la **ricerca**, il **predecessore**, il **successore**, il **minimo**, il **massimo**, l'**inserimento** e la **cancellazione**, in un tempo proporzionale ad  $h$ , ovvero  $O(h)$ .
- Queste azioni risultano efficienti quando l'albero è di bassa altezza. Tuttavia, **se l'albero diventa troppo alto, le prestazioni di queste operazioni possono essere comparabili a quelle ottenibili con una lista concatenata.**
- Per ovviare a questo problema, esistono varie tecniche per mantenere gli alberi di ricerca **equilibrati**. Tra queste, gli alberi red-black rappresentano una soluzione efficace, assicurando che le operazioni sopra menzionate possano essere realizzate in un tempo  $O(\log n)$  nel caso peggiore, dove  $n$  è il numero di **elementi nell'albero**.

## Definizione formale:

- Un **albero red-black** è un albero binario di ricerca con un bit aggiuntivo di memoria per ogni nodo: il colore del nodo, che può essere ROSSO (red) o NERO (black).
- Assegnando dei vincoli al modo in cui i nodi possono essere colorati lungo qualsiasi percorso che va dalla radice a una foglia, gli alberi red-black garantiscono che nessuno di tali percorsi sia più di due volte più lungo di qualsiasi altro, quindi l'albero è approssimativamente **bilanciato**. Ogni nodo dell'albero adesso contiene i campi **color, key, left, right e p**. Se manca un figlio o il padre di un nodo, il corrispondente campo puntatore del nodo contiene il valore NIL.

# Un albero binario di ricerca è un albero rosso nero se soddisfa le seguenti proprietà:

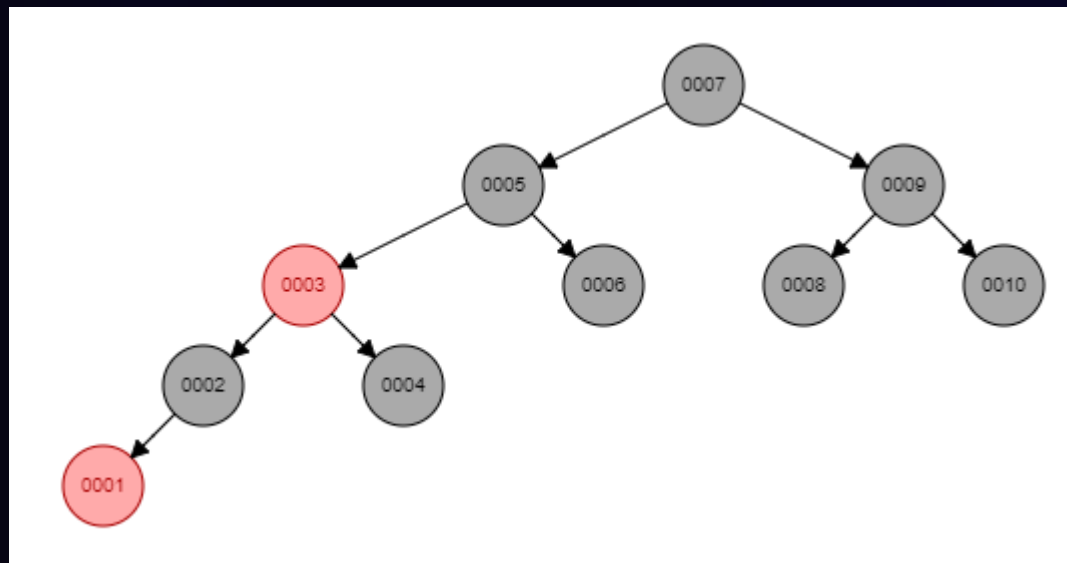
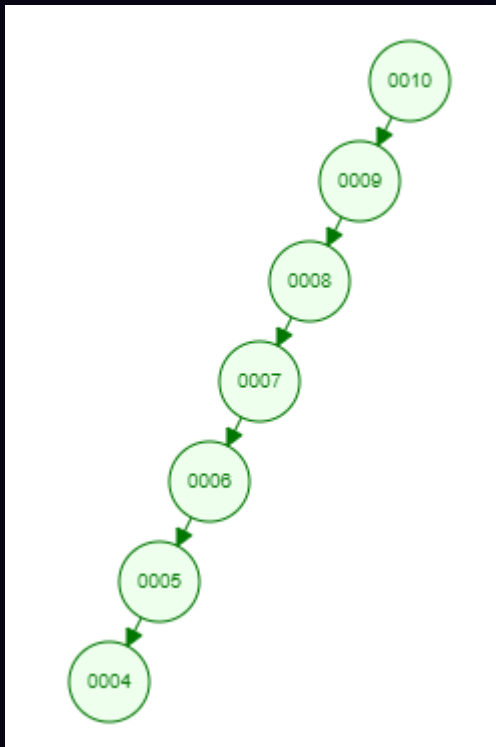
1. Ogni nodo è rosso o nero.
2. La radice è nera.
3. Ogni foglia (NIL) è nera.
4. Se un nodo è rosso, allora entrambi i suoi figli sono neri.
5. Per ogni nodo, tutti i percorsi che vanno dal nodo alle foglie discendenti contengono lo stesso numero di nodi neri.



# Definizioni importanti:

- Definiamo **altezza nera** di un nodo  $x$ , indicata da  $bh(x)$ , il numero di nodi neri lungo un percorso che inizia dal nodo  $x$  (ma non lo include) e finisce in una **foglia**. Per la proprietà 5, il concetto di altezza nera è ben definito, in quanto tutti i percorsi che scendono dal nodo hanno lo stesso numero di nodi neri.
- L'**altezza nera** di un albero red-black è l'altezza nera della sua radice.
- Otteniamo quindi che l'altezza massima di un albero red-black con  $n$  nodi interni è  $2 \lg(n + 1)$

Una immediata conseguenza è che le operazioni sugli insiemi dinamici SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR possono essere implementate nel tempo  $O(\lg n)$  negli alberi red-black, perché possono essere eseguite nel tempo  $O(h)$  in un albero di ricerca di altezza  $h$  e qualsiasi albero red-black di  $n$  nodi è un albero di ricerca di altezza  $O(\lg n)$

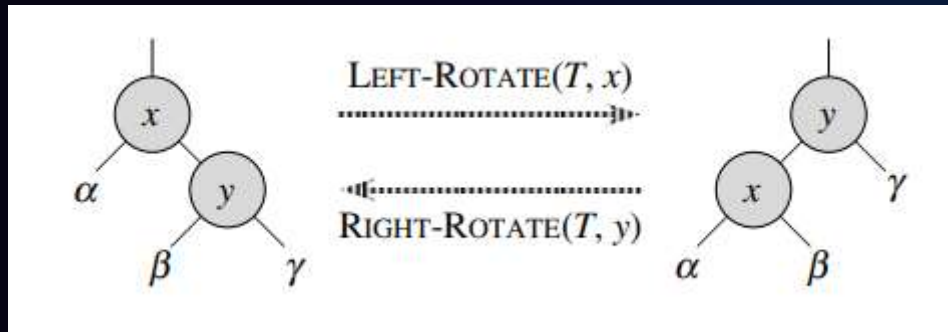


Differenza di inserimento delle chiavi  
10,9,8,7,6,5,4,3,2,1 in un albero binario di ricerca e  
in un albero binario di ricerca black-red

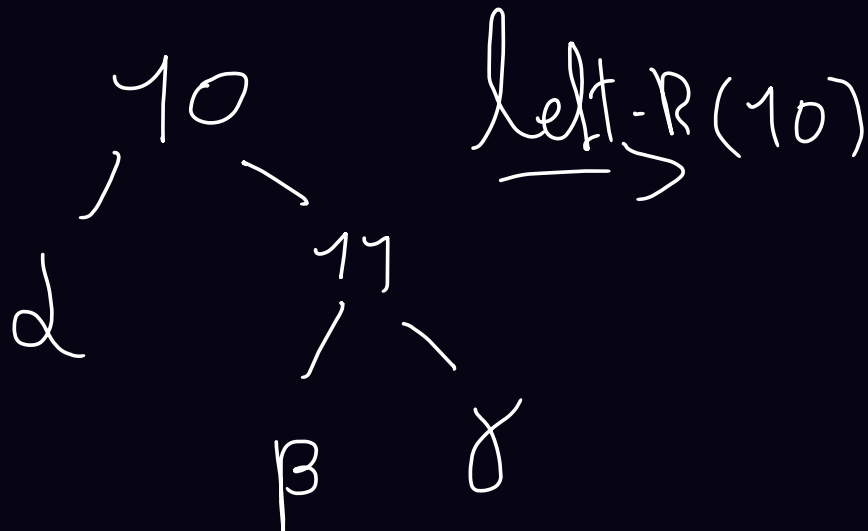


# Rotazioni

- Le operazioni di ricerca **TREE-INSERT** e **TREE-DELETE**, quando vengono eseguite in un albero red-black di  $n$  chiavi, richiedono un tempo  $O(\lg n)$ . Poiché queste operazioni modificano l'albero, il risultato potrebbe **violare le proprietà red-black** elencate. **Per ripristinare queste proprietà**, dobbiamo modificare i colori di qualche nodo dell'albero e anche la struttura dei puntatori.
- Per modificare la struttura dei puntatori viene effettuata una **rotazione**: un'operazione locale in un albero di ricerca che preserva la proprietà degli alberi binari di ricerca. Esistono due tipi di **rotazioni**: **rotazione sinistra** e **rotazione destra**



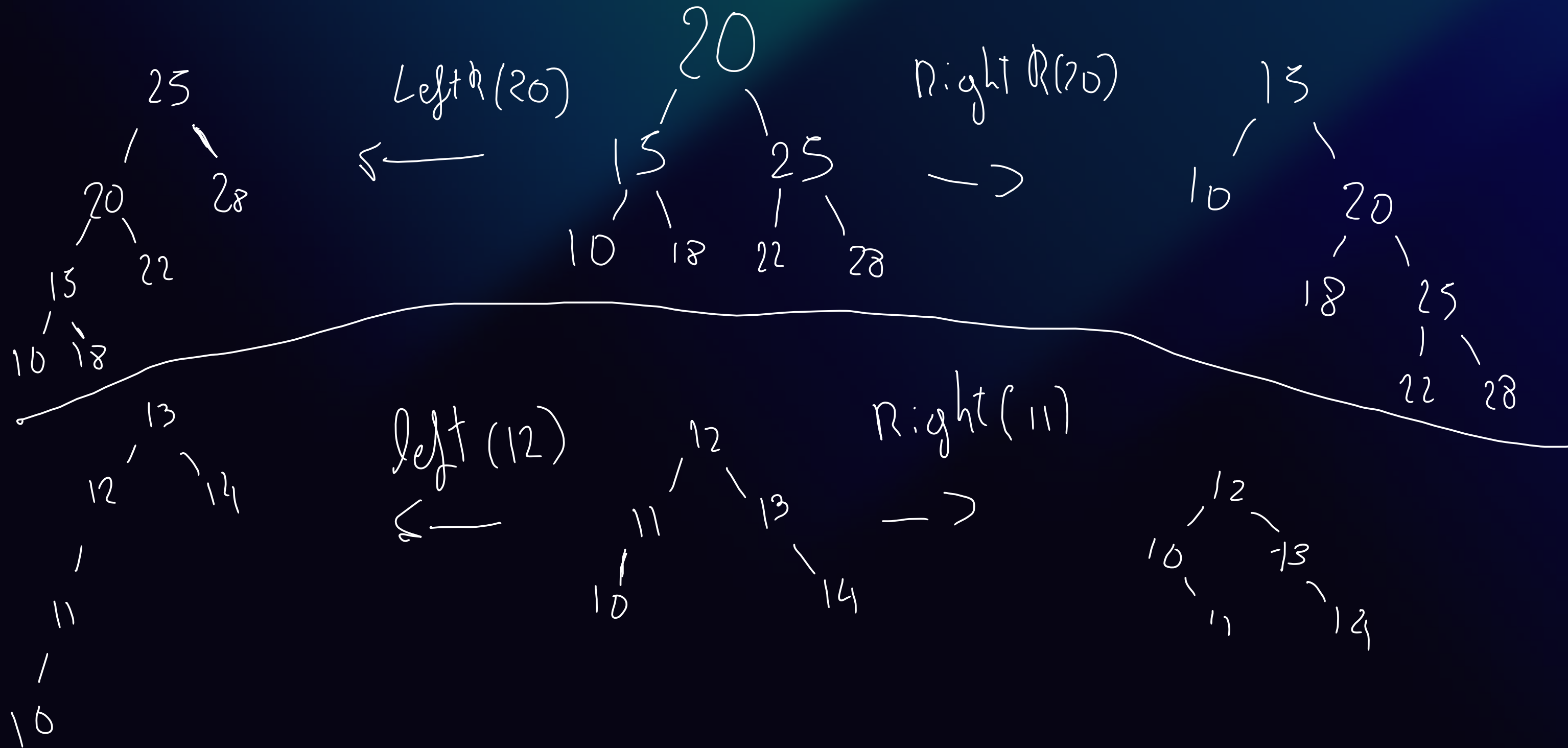
## Esempio:



Right-R(15)  
→



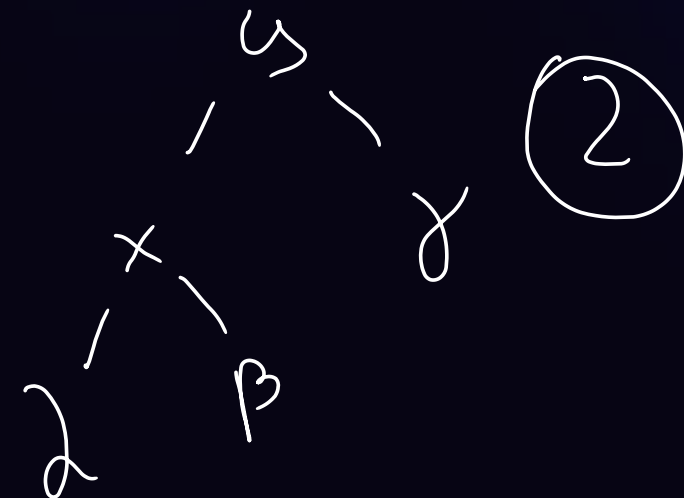
Altri esempi sulle rotazioni:



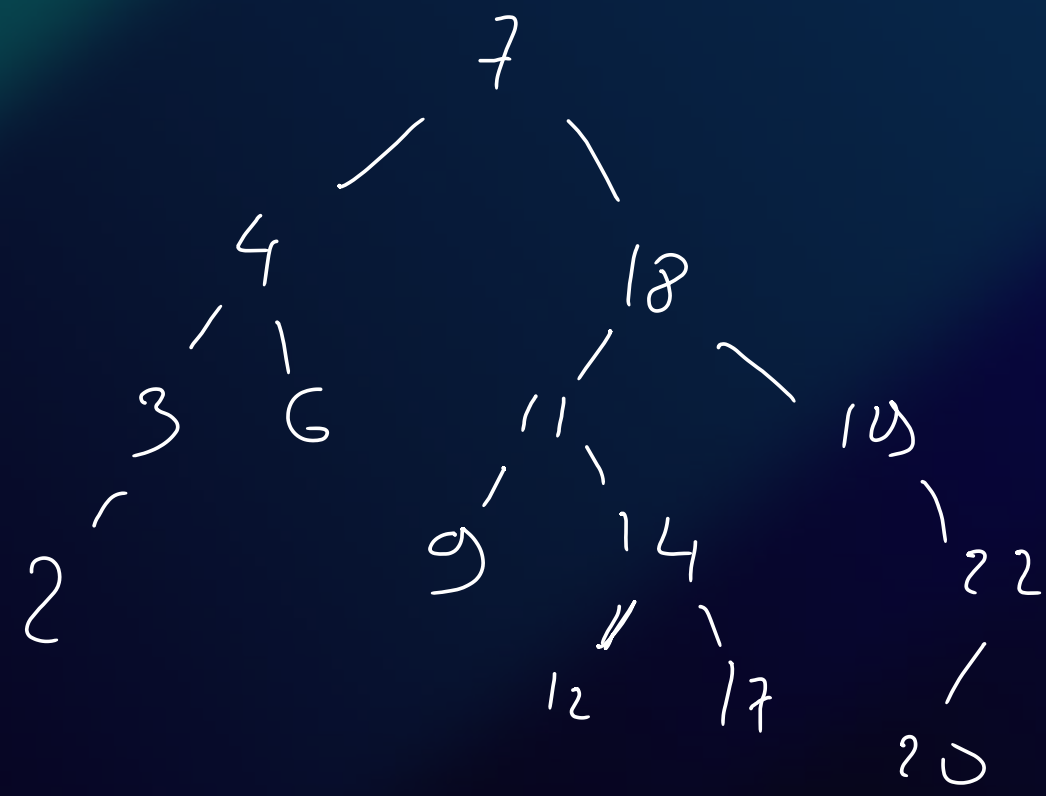
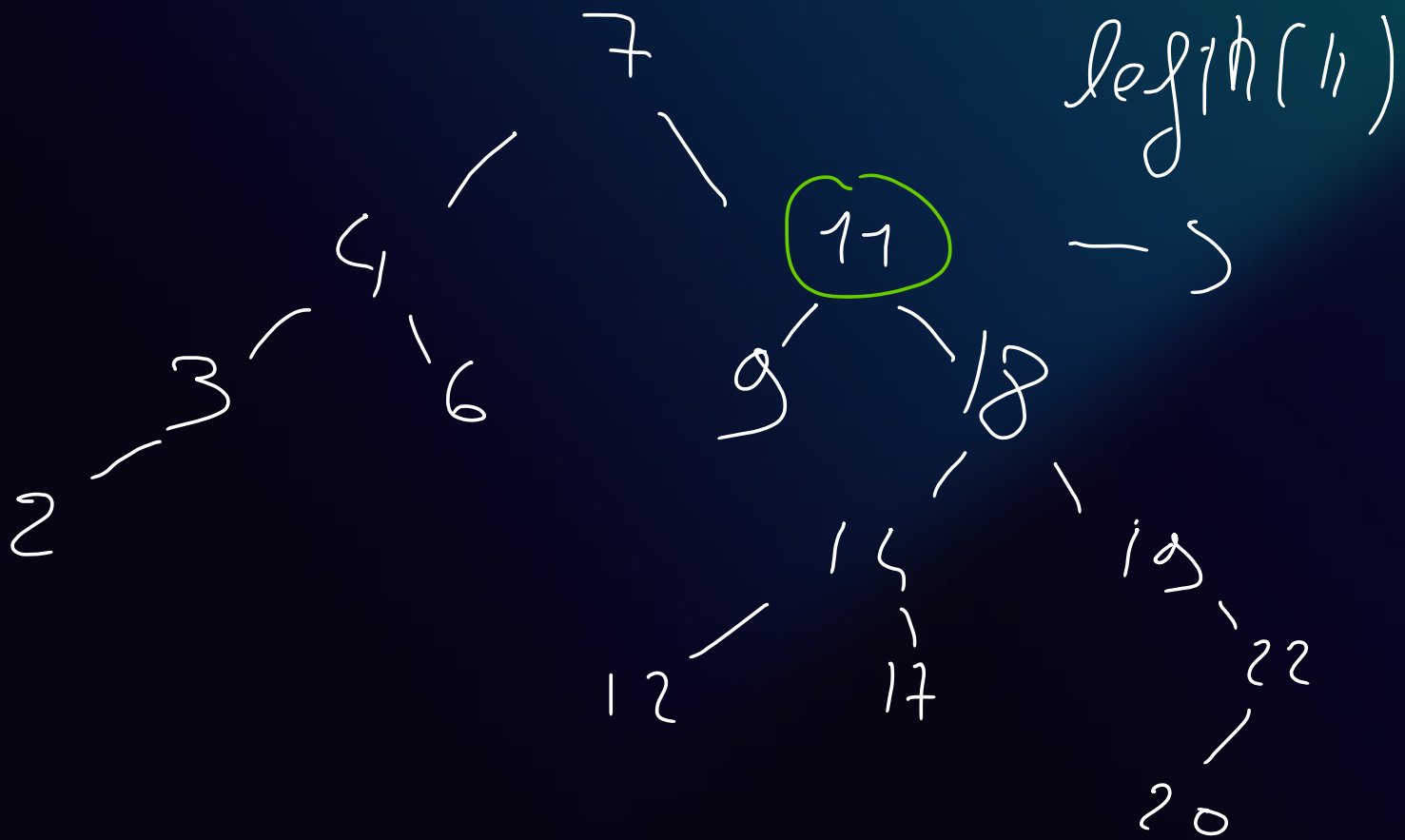
## Più difficile a dirsi che a farsi:

LEFT-ROTATE( $T, x$ )

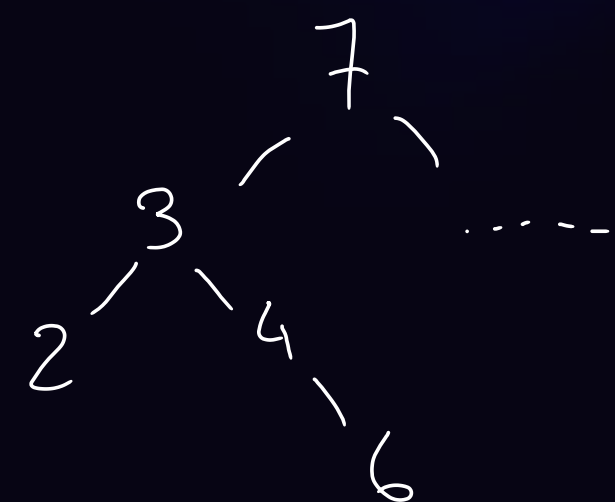
```
1   $y \leftarrow \text{right}[x]$            ▷ Imposta  $y$ .
2   $\text{right}[x] \leftarrow \text{left}[y]$    ▷ Ruota il sottoalbero sinistro di  $y$ 
                                   nel sottoalbero destro di  $x$ .
3  if  $\text{left}[y] \neq \text{nil}[T]$ 
4  then  $p[\text{left}[y]] \leftarrow x$ 
5   $p[y] \leftarrow p[x]$            ▷ Collega il padre di  $x$  a  $y$ .
6  if  $p[x] = \text{nil}[T]$             $\rightarrow x$  è la radice?
7  then  $\text{radice}[T] \leftarrow y$     $\rightarrow y$  nuova radice
8  else if  $x = \text{left}[p[x]]$ 
9  then  $\text{left}[p[x]] \leftarrow y$ 
10 else  $\text{right}[p[x]] \leftarrow y$ 
11  $\text{left}[y] \leftarrow x$            ▷ Pone  $x$  a sinistra di  $y$ .
12  $p[x] \leftarrow y$ 
```



Altri esempi sulle rotazioni:



RightR(4)





# Inserimento:

Per inserire un nodo  $z$  nell'albero  $T$  si inserisce come se fosse un normale albero binario di ricerca e poi coloriamo  $z$  di rosso. Per garantire che le proprietà red-black siano preservate, chiamiamo una procedura ausiliaria **RB-INSERT-FIXUP** per ricolorare i nodi ed effettuare le rotazioni. La chiamata **RB-INSERT( $T, z$ )** inserisce il nodo  $z$ , il cui campo key si suppone sia stato già riempito, nell'albero red-black  $T$ .

```
RB-INSERT( $T, z$ )
1   $y \leftarrow nil[T]$ 
2   $x \leftarrow radice[T]$ 
3  while  $x \neq nil[T]$ 
4      do  $y \leftarrow x$ 
5          if  $key[z] < key[x]$ 
6              then  $x \leftarrow left[x]$ 
7              else  $x \leftarrow right[x]$ 
8   $p[z] \leftarrow y$ 
9  if  $y = nil[T]$ 
10     then  $radice[T] \leftarrow z$ 
11     else if  $key[z] < key[y]$ 
12         then  $left[y] \leftarrow z$ 
13         else  $right[y] \leftarrow z$ 
14   $left[z] \leftarrow nil[T]$ 
15   $right[z] \leftarrow nil[T]$ 
16   $color[z] \leftarrow \text{ROSSO}$ 
17  RB-INSERT-FIXUP( $T, z$ )
```

Ci sono quattro differenze fra le procedure **TREE-INSERT** e **RB-INSERT**.

- tutte le istanze di NIL in **TREE-INSERT** sono sostituite con  $nil[T]$ .
- impostiamo  $left[z]$  e  $right[z]$  a  $nil[T]$  nelle righe 14–15 di **RB-INSERT**, per mantenere la struttura appropriata dell'albero.
- coloriamo  $z$  di rosso nella riga 16.
- poiché colorare  $z$  di rosso può causare una violazione di una delle proprietà red-black, chiamiamo **RB-INSERT-FIXUP( $T, z$ )** nella riga 17 di **RB-INSERT** per ripristinare le proprietà red-black.

## RB-INSERT-FIXUP:

Quali proprietà red-black possono essere violate prima della chiamata di RBINSERT-FIXUP?

- La proprietà 1 certamente continua a essere valida, come la proprietà 3, in quanto entrambi i figli del nuovo nodo rosso che è stato inserito sono la sentinella nil[T]. Anche la proprietà 5, secondo la quale il numero di nodi neri è lo stesso in ogni percorso da un dato nodo, è soddisfatta perché il nodo z sostituisce la sentinella (nera), e il nodo z è rosso con figli sentinella.
- Quindi, le uniche proprietà che potrebbero essere violate sono la proprietà 2, che richiede che la radice sia nera, e la proprietà 4, che dice che un nodo rosso non può avere un figlio rosso. Entrambe le possibili violazioni si verificano se z è rosso.
- Sistemiamo l'albero analizzando 3 casi diversi: lo zio y di z è rosso, lo zio y di z è nero e z è un figlio destro, lo zio y di z è nero e z è un figlio sinistro

**Caso 1: lo zio y di z è rosso**  $ins(L_1)$



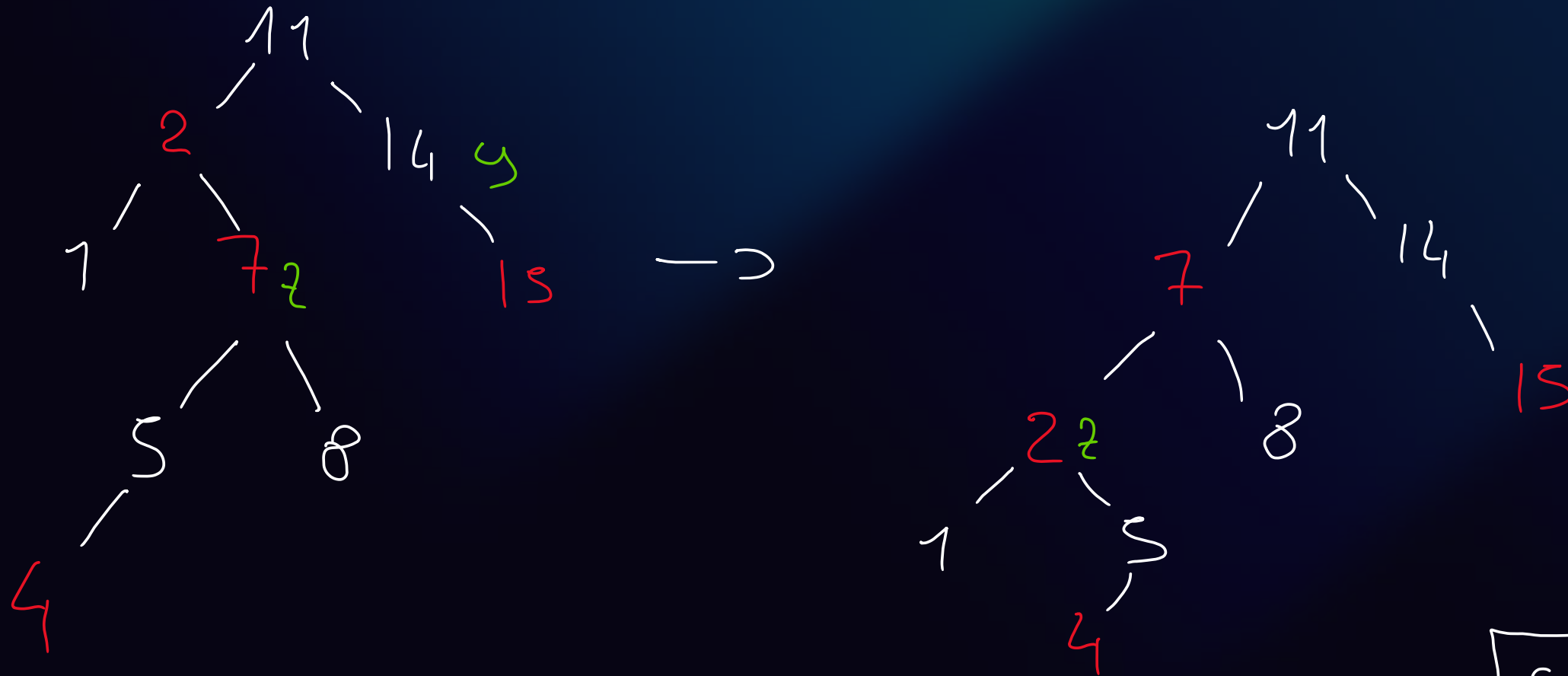
(1)  $z \rightarrow \text{rosso}$   
 $P[z] \rightarrow \text{rosso}$

$\hookrightarrow \text{zio } y \rightarrow \text{rosso}$

Caso 1: y e  $P[z]$  diventano neri:  
 $P[P[z]]$  diventa rosso  
 $P[P[z]]$  diventa la nuova z

## RB-INSERT-FIXUP:

Caso 2: lo zio y di z è nero e z è un figlio destro



Ci sono ancora violazioni?

Se sì, quali?

Caso 2:

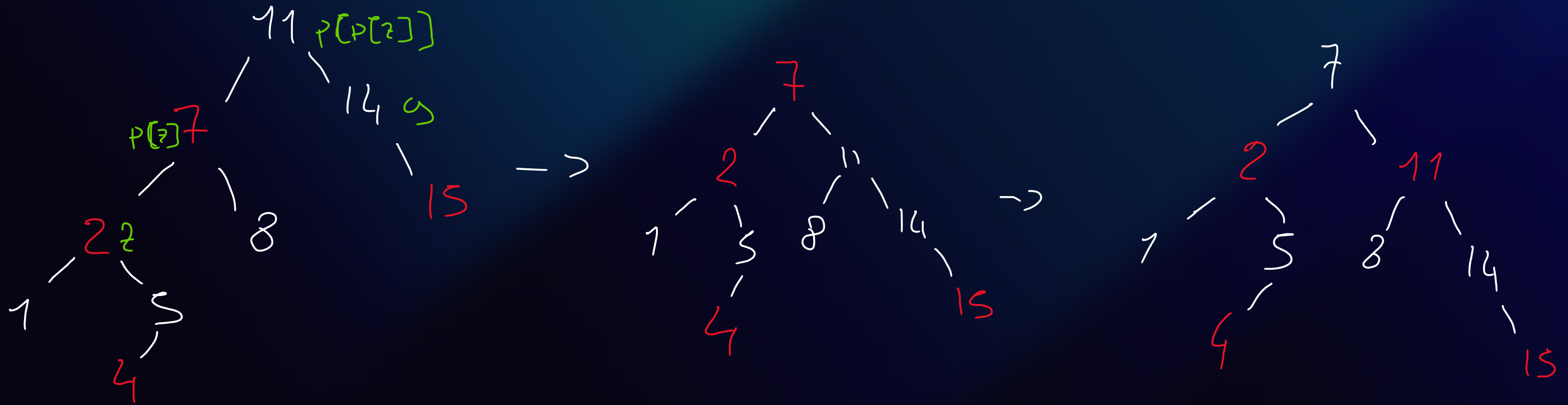
zuo ↑ p a sinistra

Caso 3

## RB-INSERT-FIXUP:

Caso 3: lo zio  $y$  di  $z$  è nero e  $z$  è un figlio sinistro  $\rightarrow$

rotazione a destra ol:  $P[P[z]]$   
e sistemare i colori:



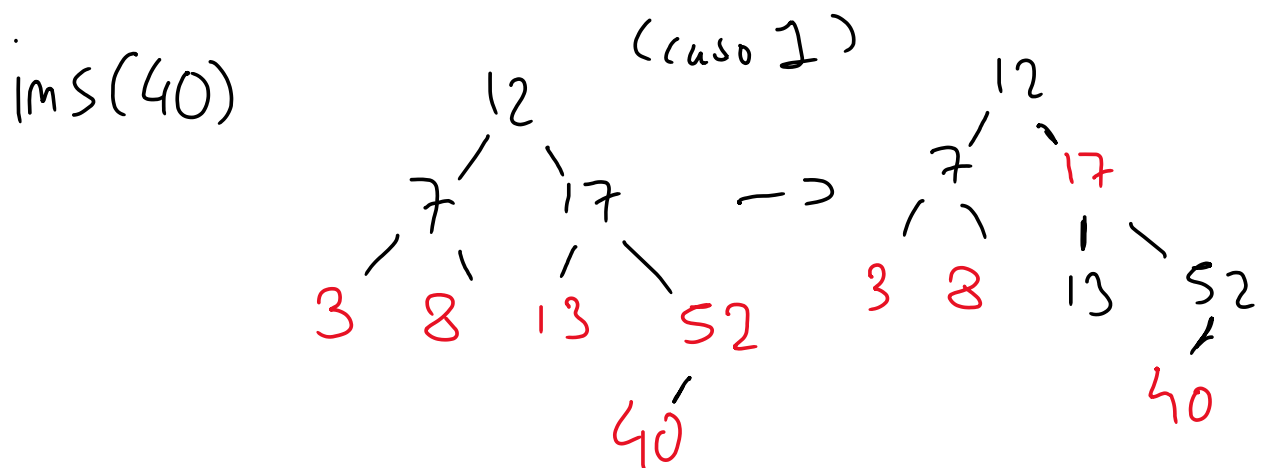
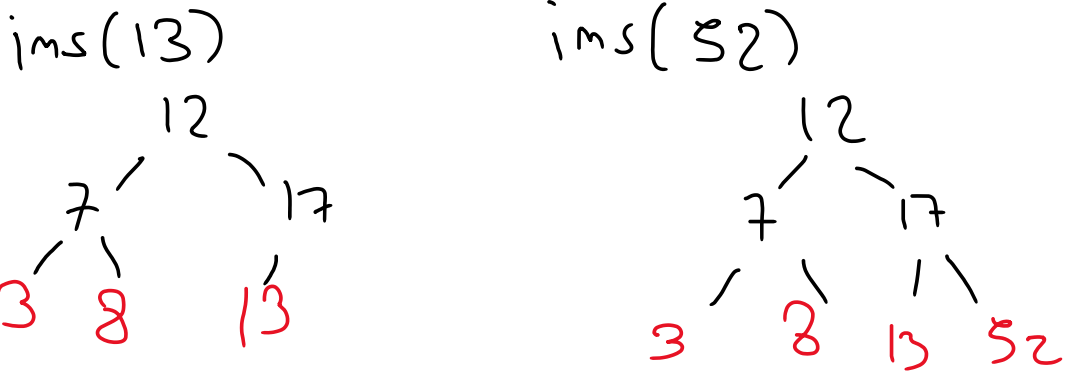
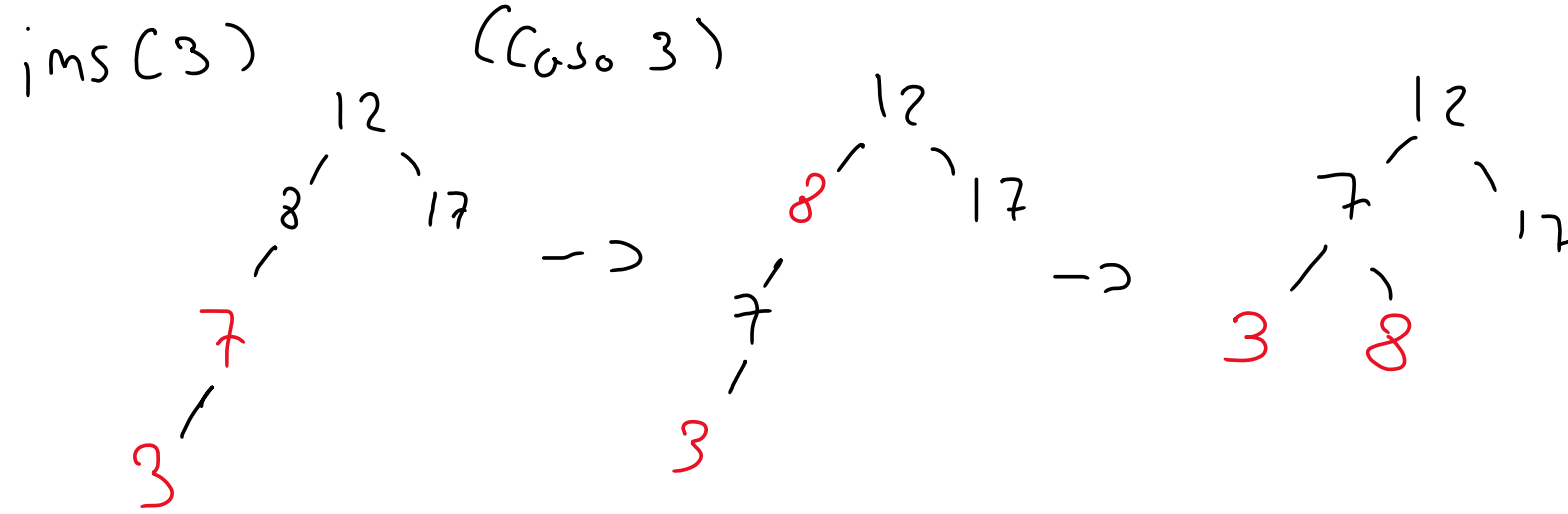
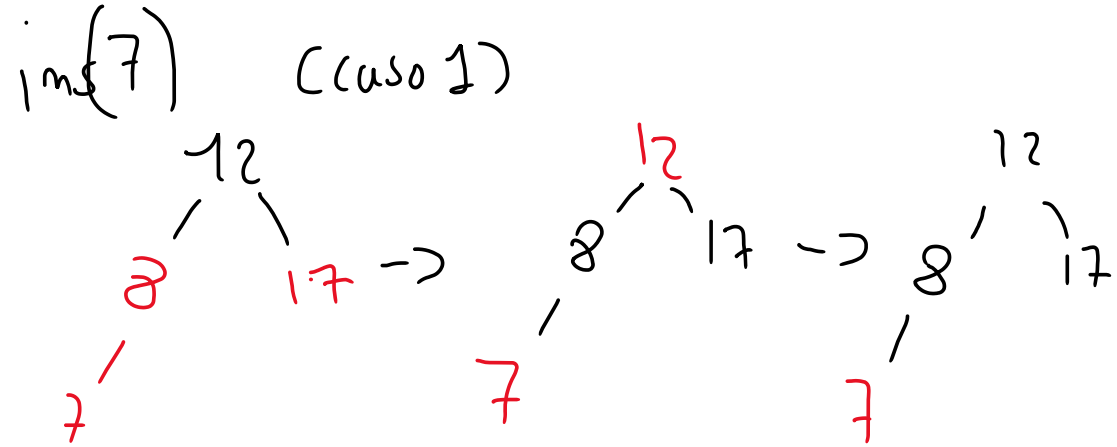
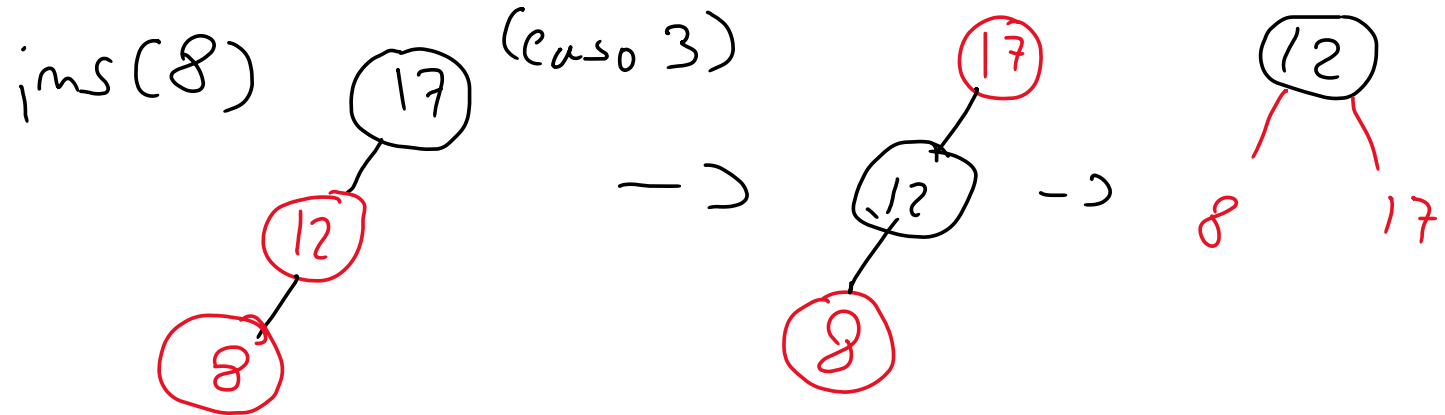
Caso 3:

- $P[z] \rightarrow$  diventa nero
- $P[P[z]] \rightarrow$  diventa rosso
- rotazione ol:  $P[P[z]]$  a destra

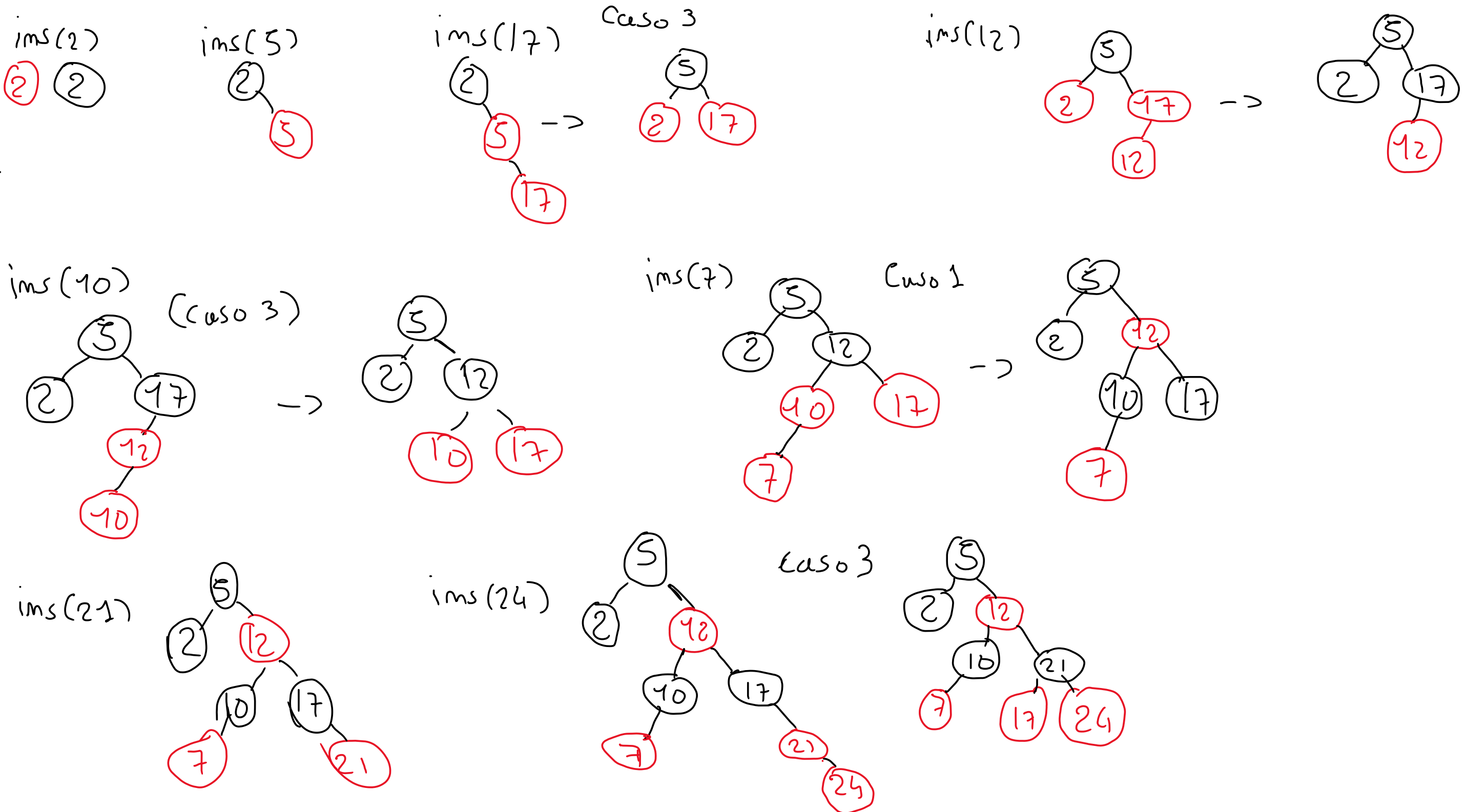


# Esercizi Inserimento:

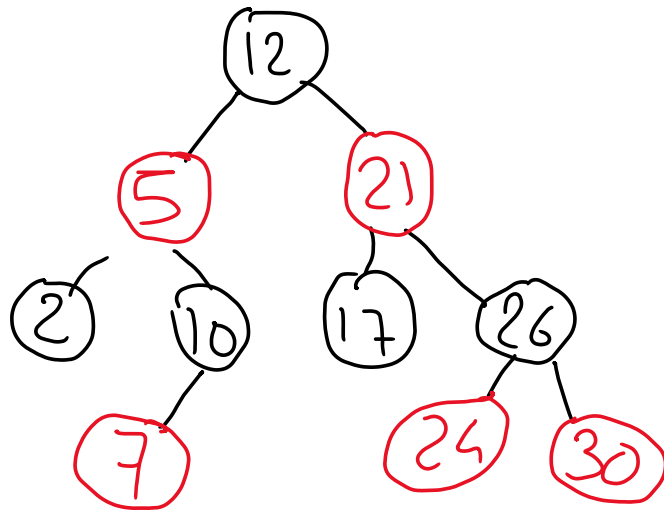
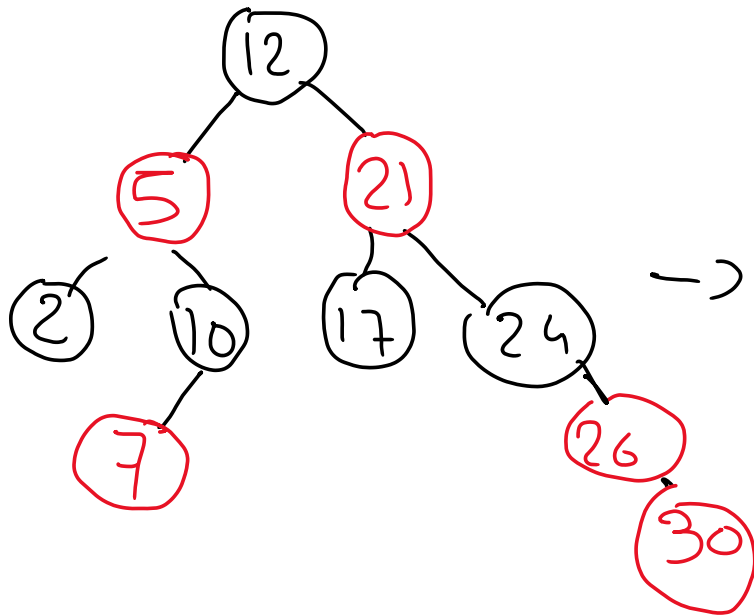
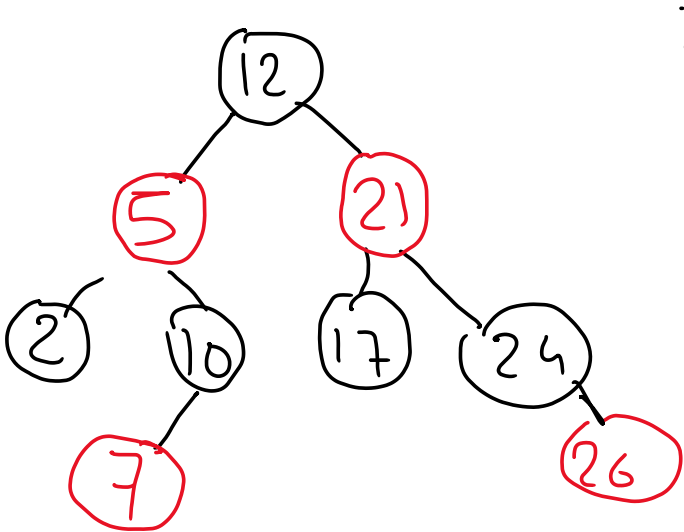
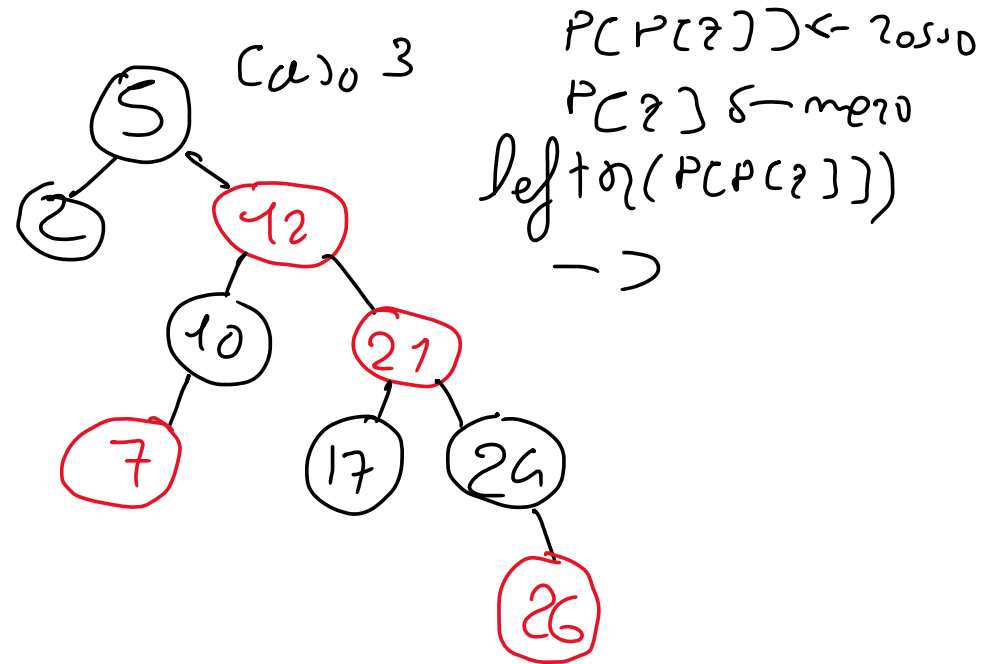
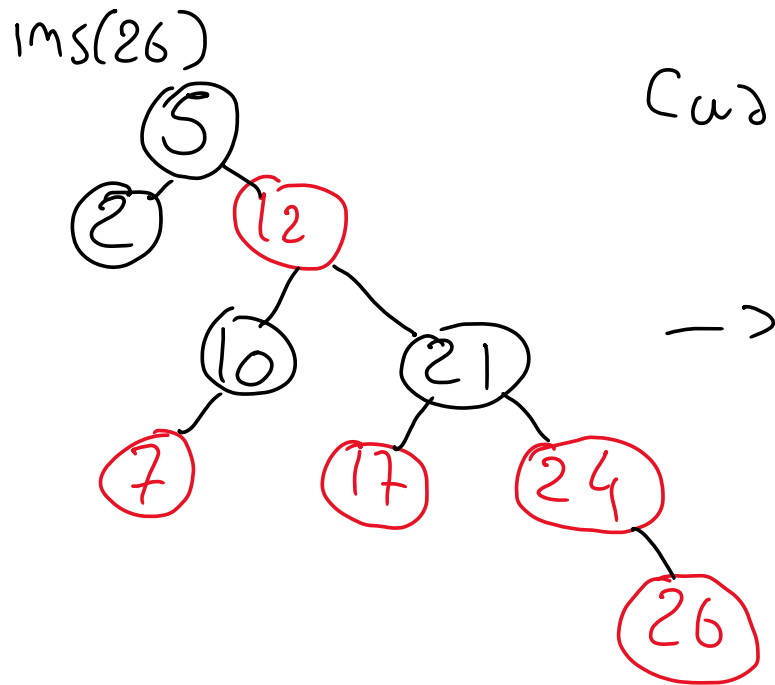
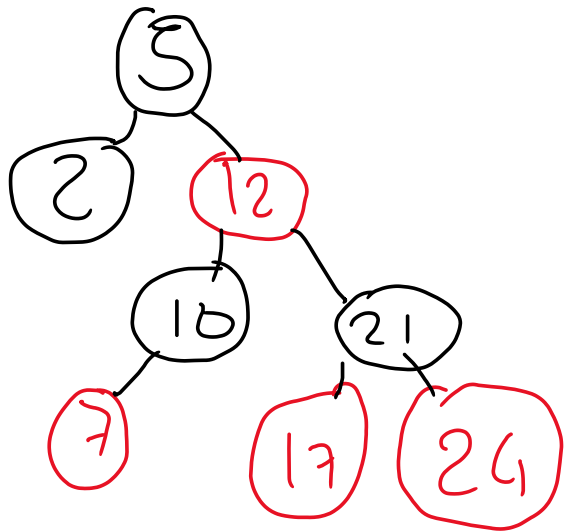
Si supponga di operare su di un albero Rosso-Nero, inizialmente vuoto, inserendo le seguenti chiavi nell'ordine dato: <17, 12, 8, 7, 3, 13, 52, 40>



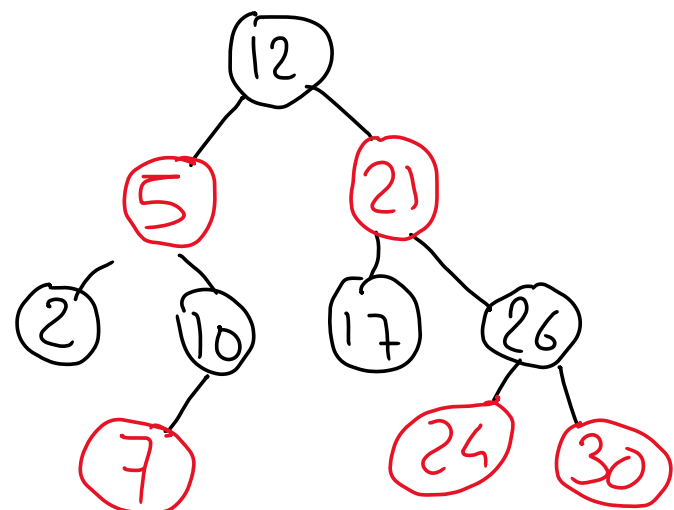
Si supponga di operare su di un albero rosso-nero inizialmente vuoto. Si effettuino l'inserimento delle seguenti chiavi, nell'ordine dato: 2, 5, 17, 12, 10, 7, 21, 24, 26, 30, 33, 31. Si fornisca la struttura dell'albero dopo ciascuna delle operazioni di inserimento.



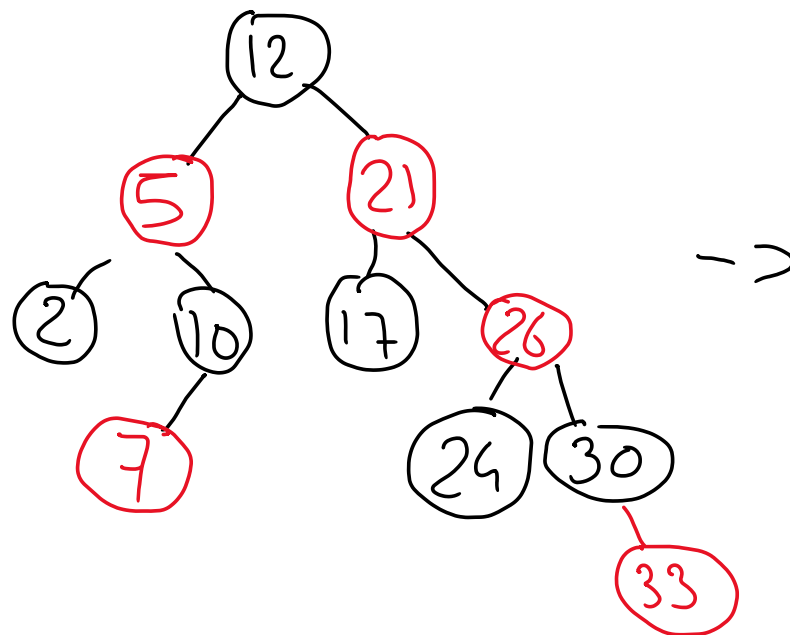
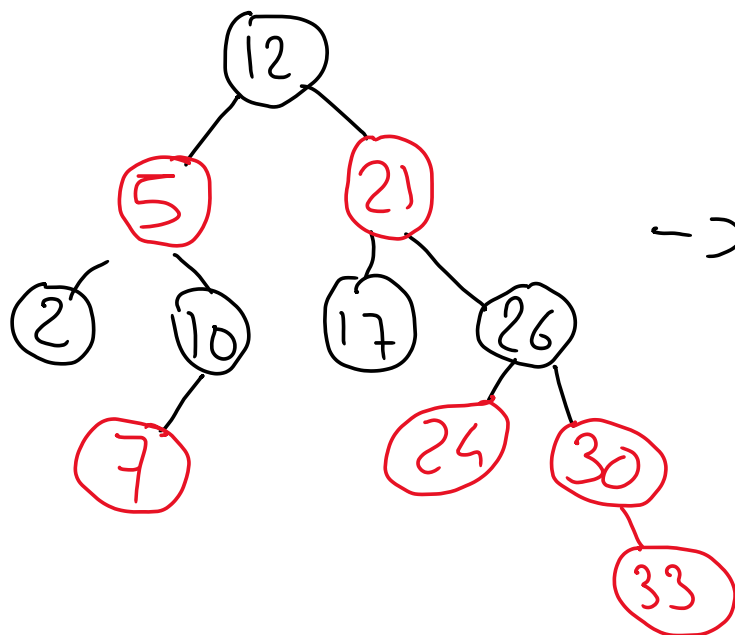
Si supponga di operare su di un albero rosso-nero inizialmente vuoto. Sieffettui l'inserimento delle seguenti chiavi, nell'ordine dato: 2, 5, 17, 12, 10, 7, 21, 24, 26, 30, 33, 31. Si fornisca la struttura dell'albero dopo ciascuna delle operazioni di inserimento.



Si supponga di operare su di un albero rosso-nero inizialmente vuoto. Sieffettui l'inserimento delle seguenti chiavi, nell'ordine dato: 2, 5, 17, 12, 10, 7, 21, 24, 26, 30, 33, 31. Si fornisca la struttura dell'albero dopo ciascuna delle operazioni di inserimento.

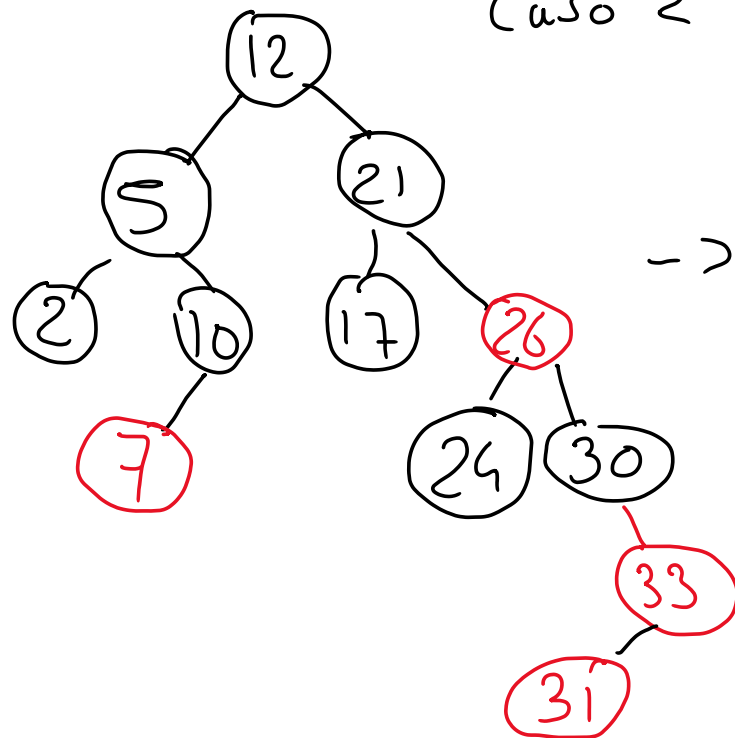


ins(33)

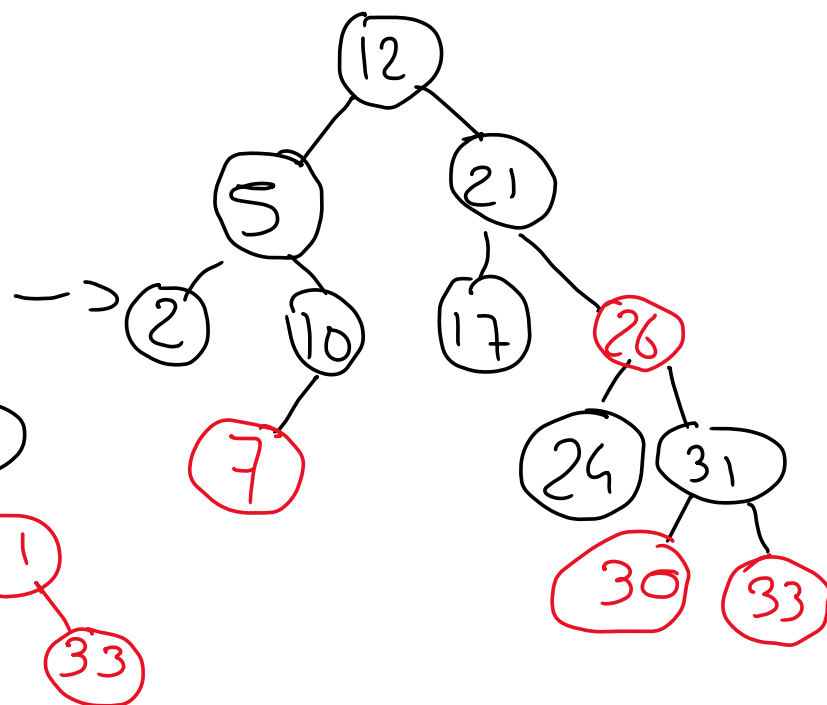


ins(31)

Caso 2



Caso 3



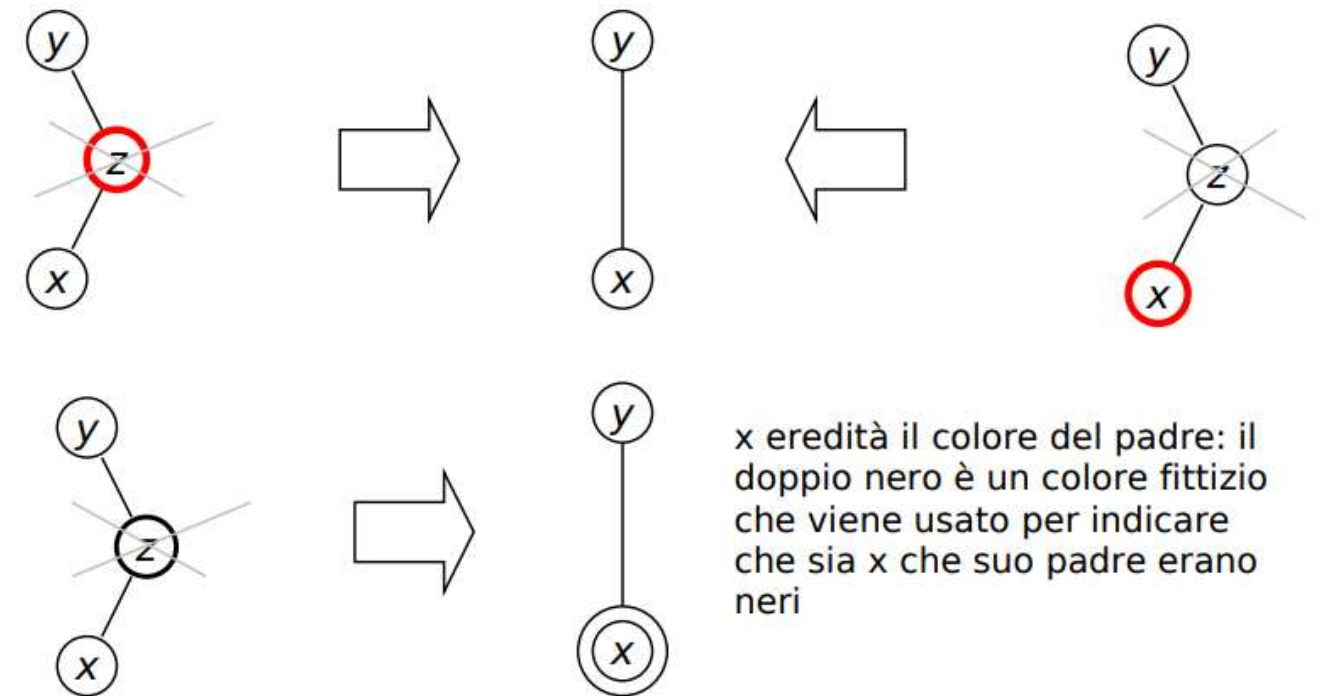
# Cancellazione

Assunzione: possiamo sempre assumere di eliminare un nodo che ha al massimo un figlio.

Infatti se dobbiamo cancellare un nodo  $z$  con due figli, possiamo rimpiazzare la chiave di  $z$  con quella del suo successore  $y$ , e poi rimuovere  $y$  (che ha al più solo il figlio destro) dall'albero.

Sia  $z$  il nodo da cancellare, e siano  $x$  e  $y$  il figlio ed il padre di  $z$ . Per eliminare il nodo  $z$  eseguiamo i seguenti passi:

1. Rimuoviamo  $z$  collegando  $y$  con  $x$
2. se  $z$  era rosso allora  $y$  e  $x$  sono neri e terminiamo
3. se  $z$  era nero (possibile violazione della proprietà 5)
  - 3.1 se  $x$  è rosso allora coloriamo  $x$  di nero
  - 3.2 se  $x$  è nero allora ricoloriamo  $y$  con un colore fittizio, detto "doppio" nero, che serve a ricordarci che abbiamo collassato due nodi neri in uno violando così la proprietà 5
  - 3.3 Ripristiniamo eventuali violazioni della proprietà 5

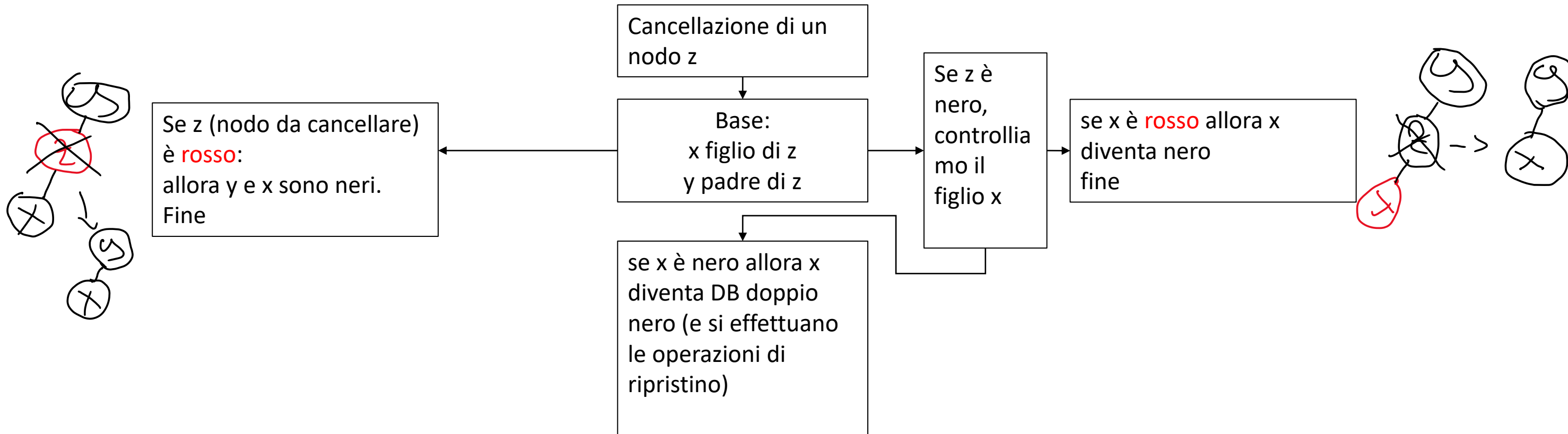




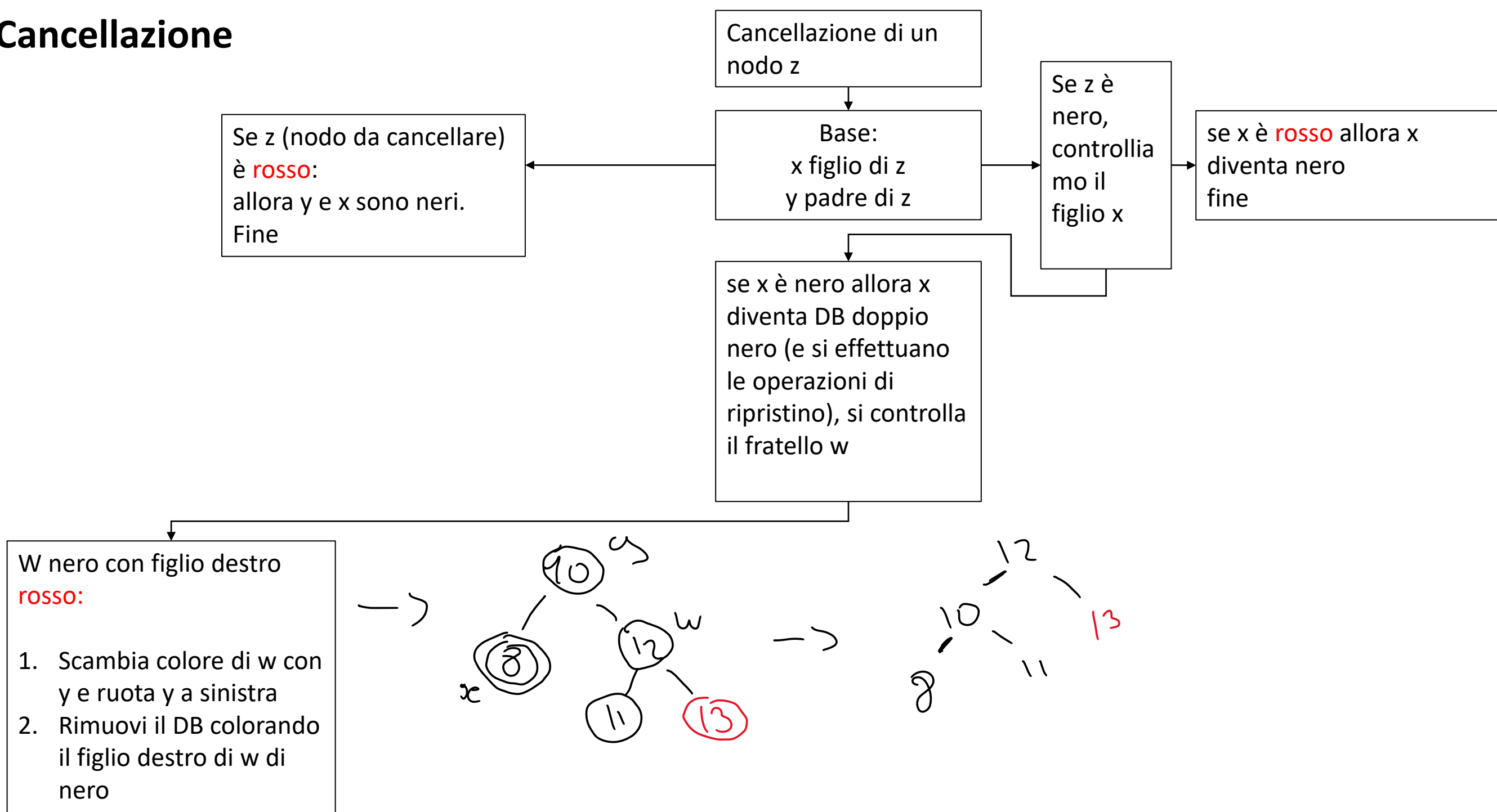
# Cancellazione

Anche in questo caso la fase di ripristino delle proprietà violate tiene conto di una serie di casi:

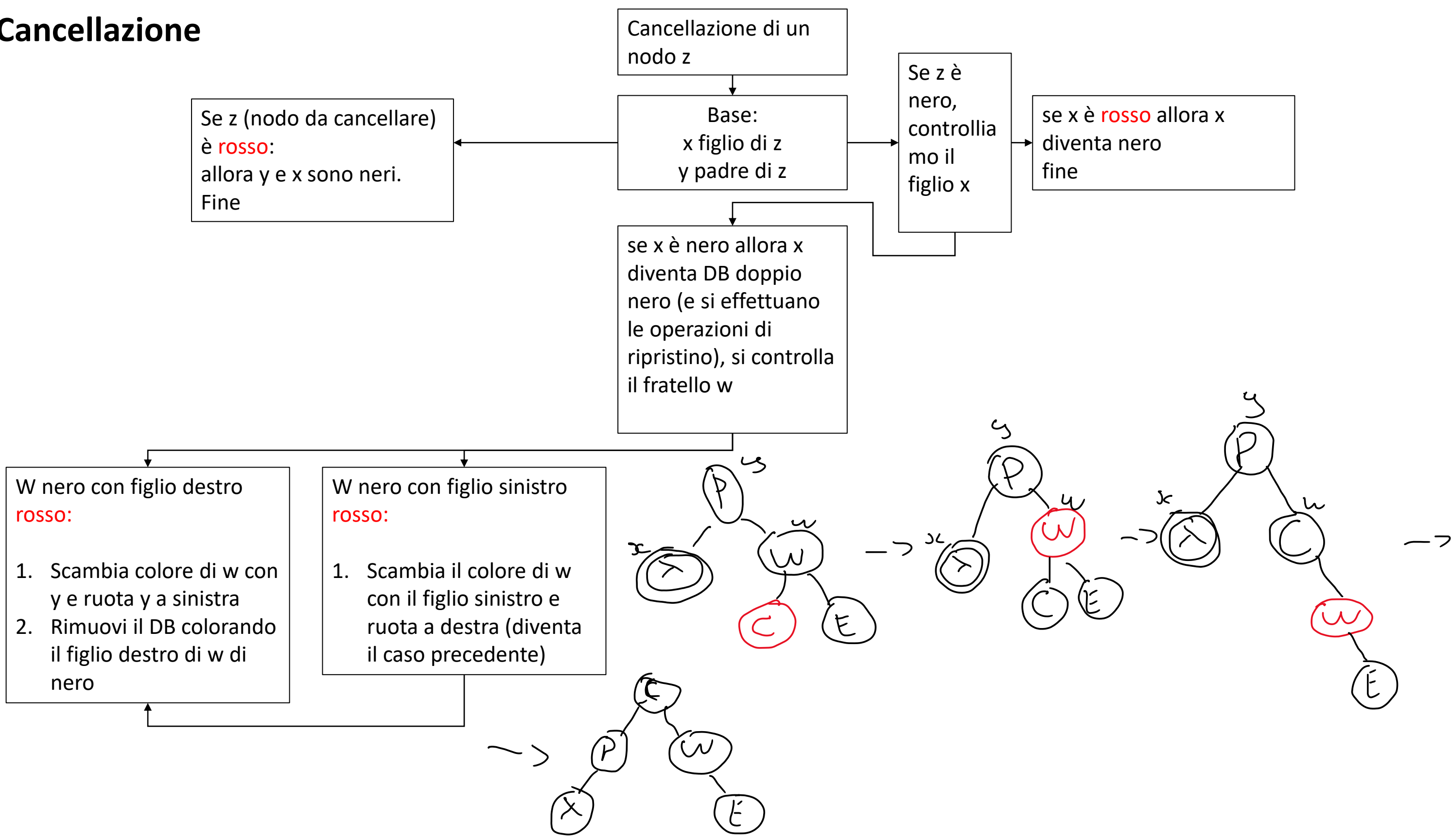
1. Il fratello w di x ha almeno un figlio rosso. Distinguiamo due possibili sottocasi:
  - 1.1 il figlio rosso di w è un figlio destro
  - 1.2 il figlio rosso di w è un figlio sinistro
2. il fratello w di x è nero ed entrambi i suoi figli sono neri
3. il fratello w di x è rosso



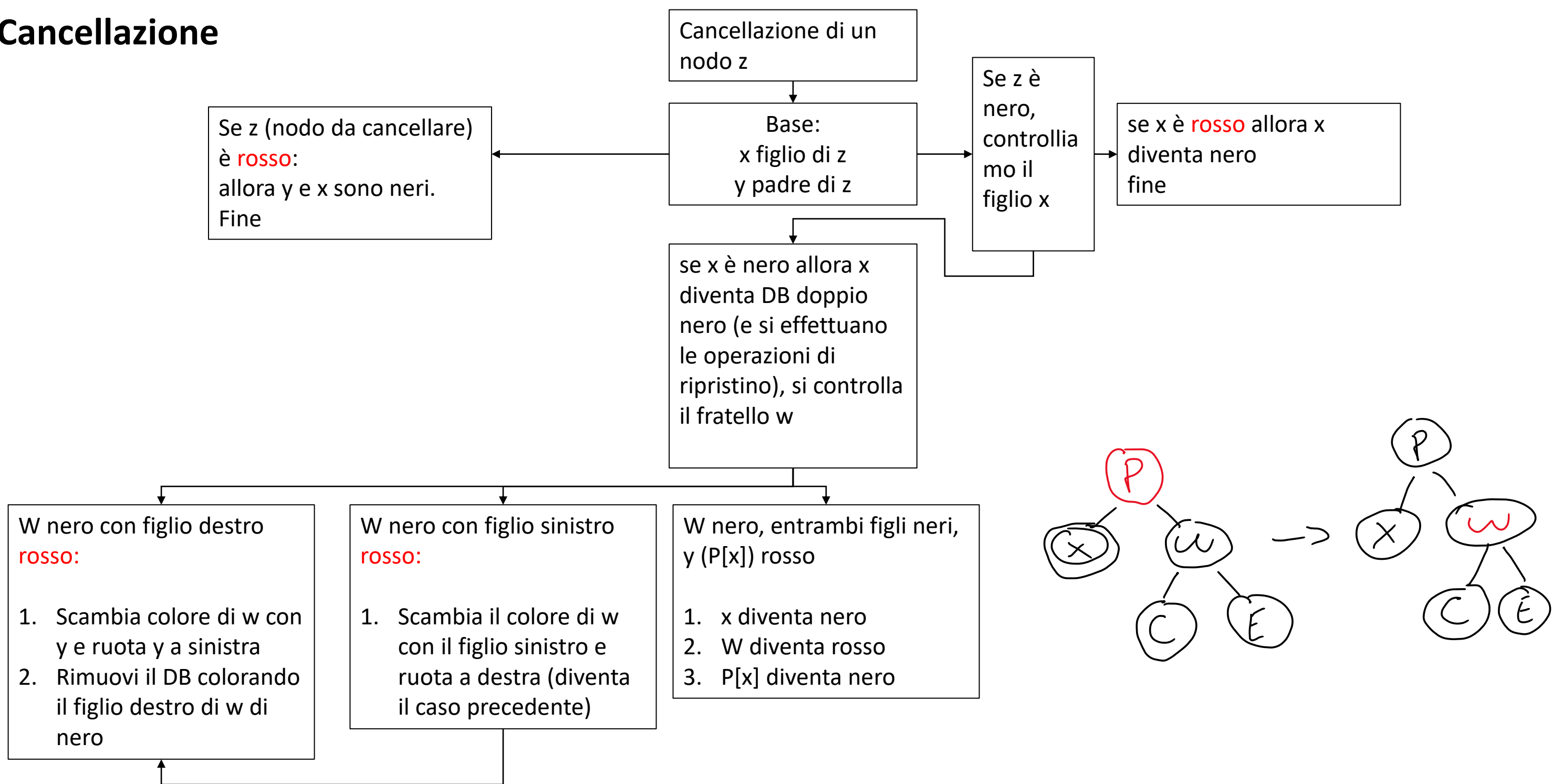
# Cancellazione



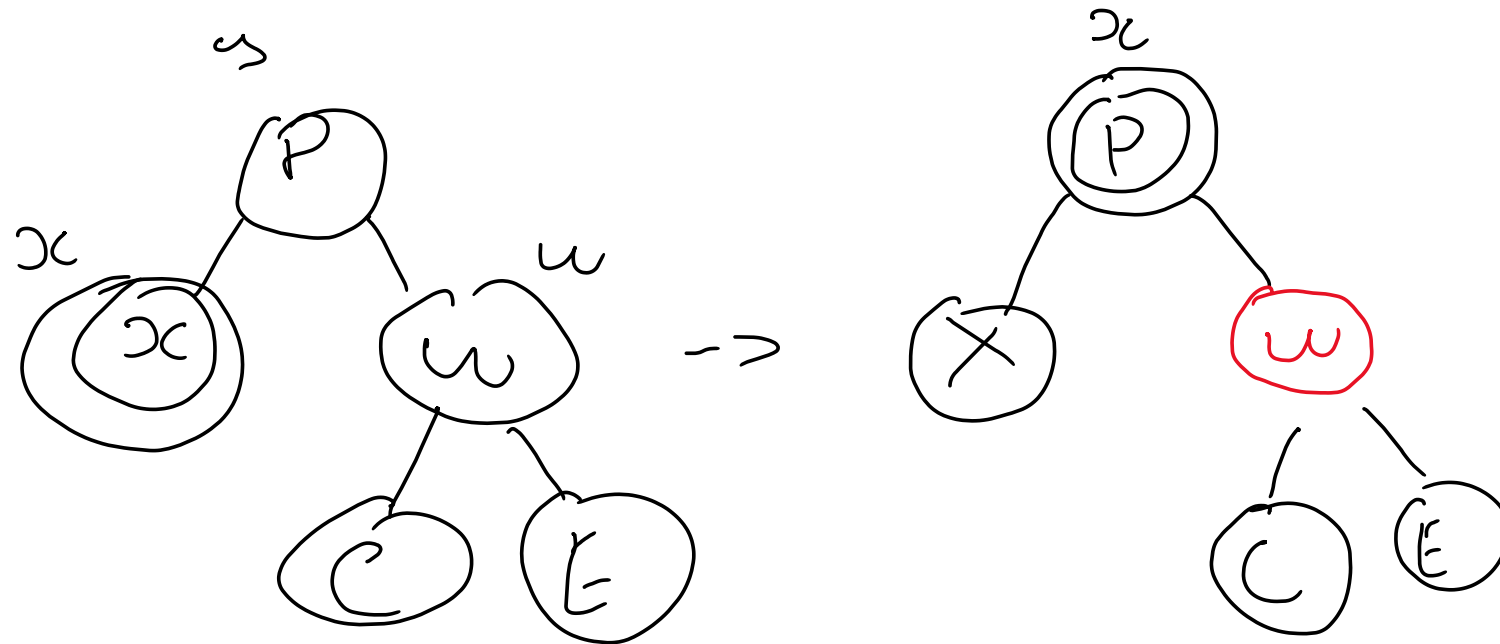
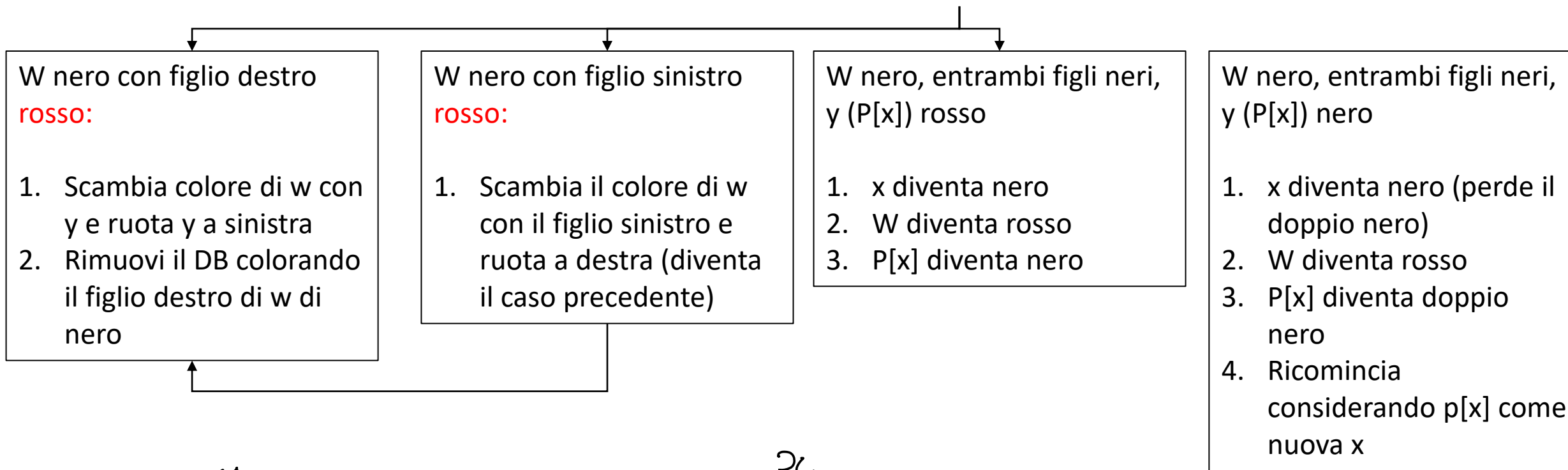
# Cancellazione



# Cancellazione

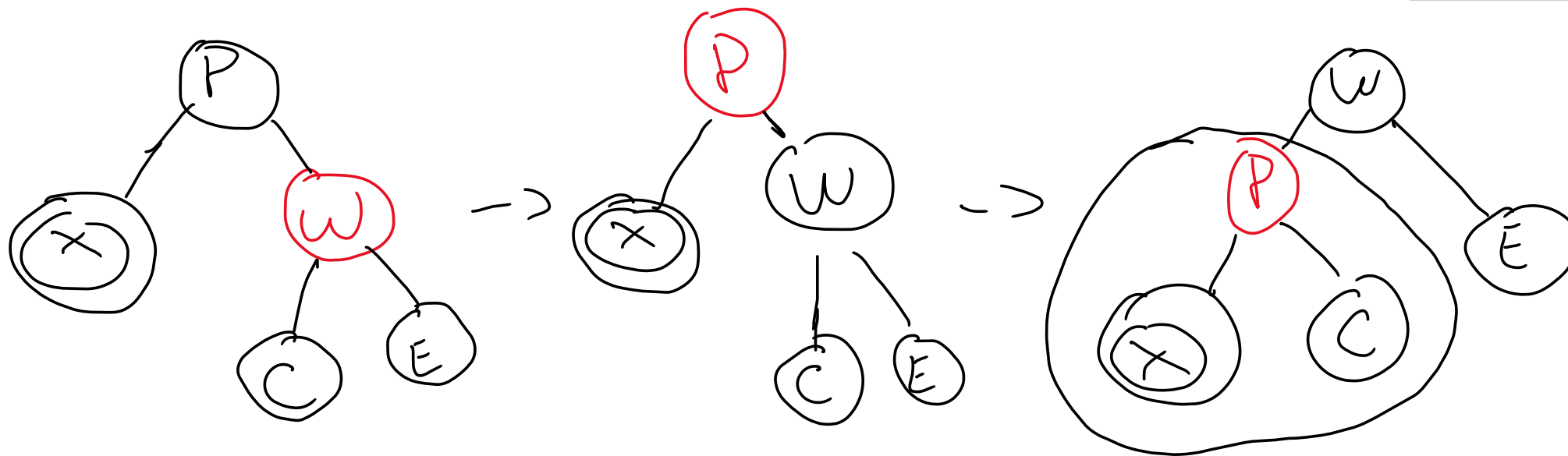
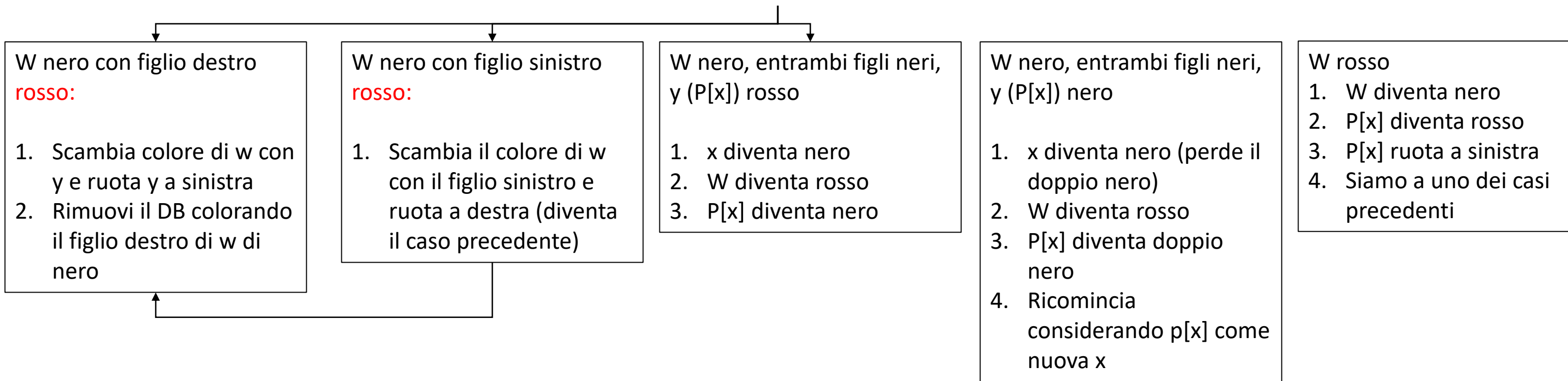


# Cancellazione

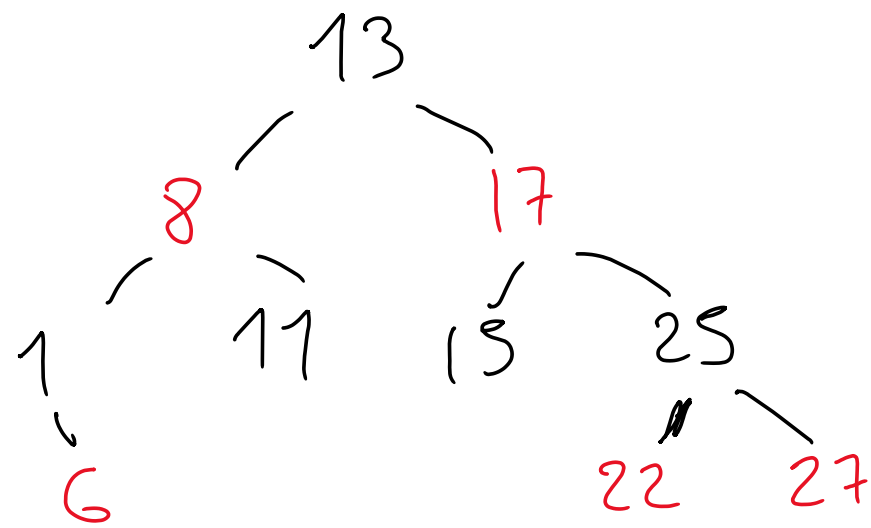




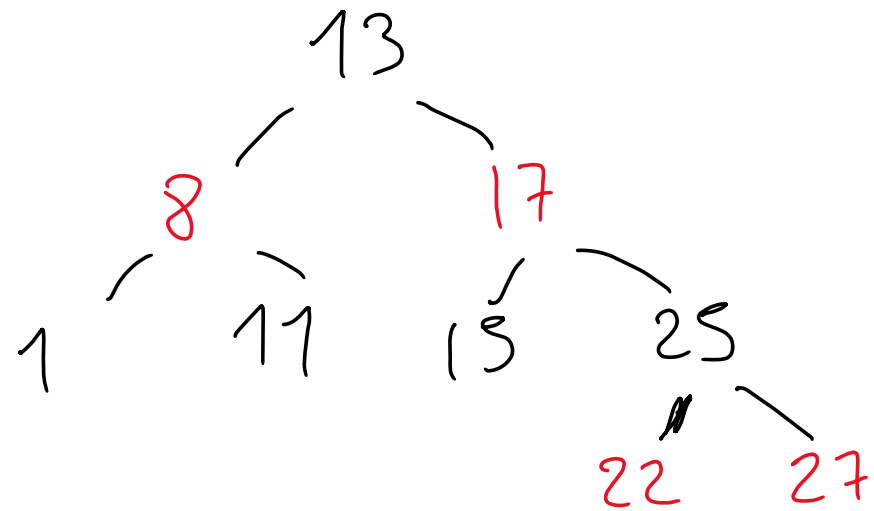
# Cancellazione



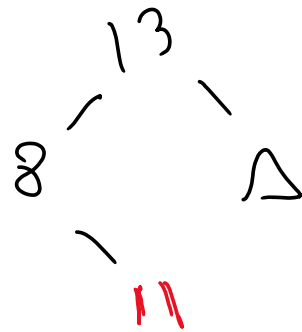
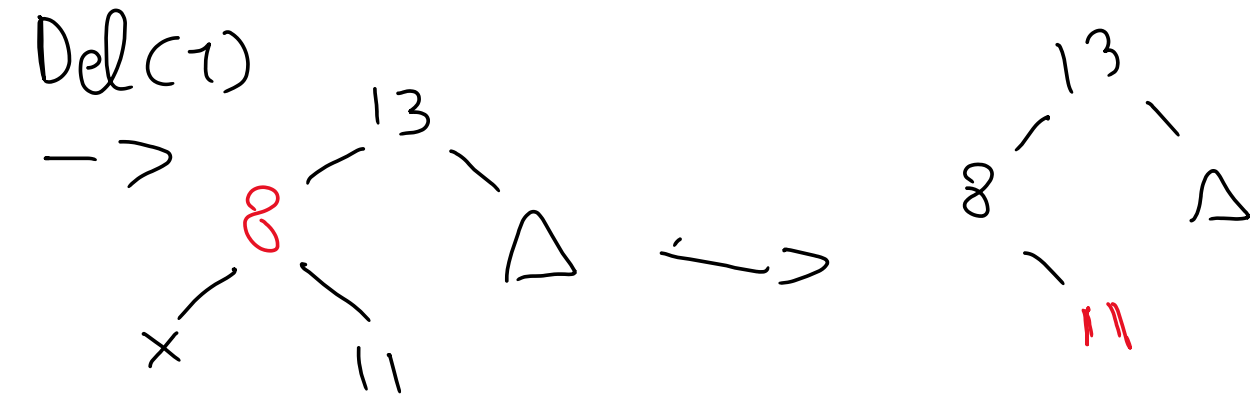
Facciamo molti esempi per capire meglio



Del(G)



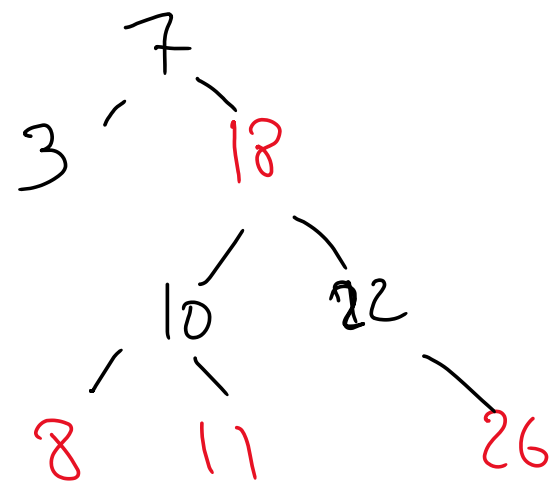
→



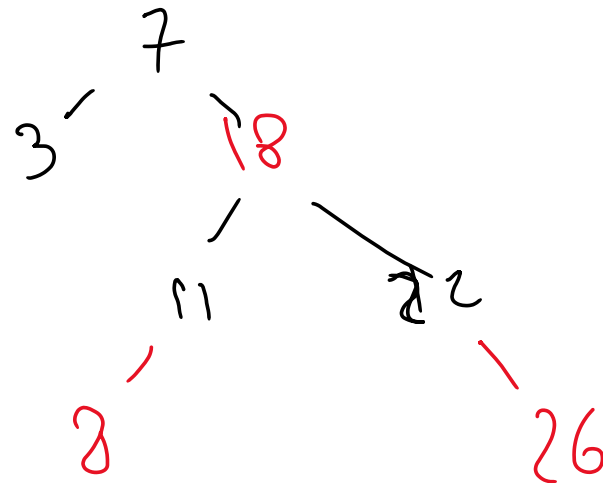
→

caso w nero, entrambi  
figli neri, p[cc] rosso

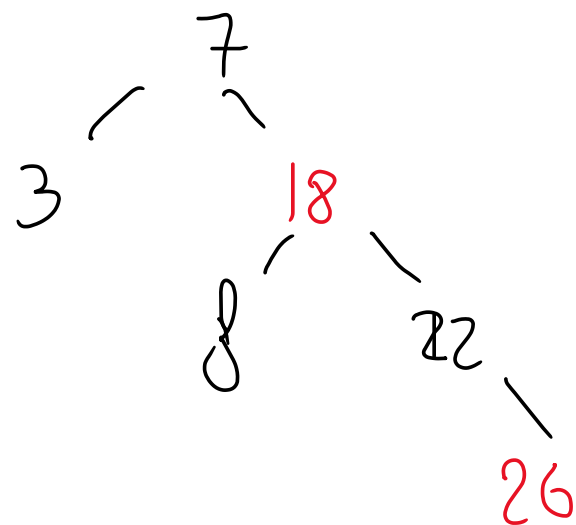
(w olivento rosso  
p[cc] olivento nero)



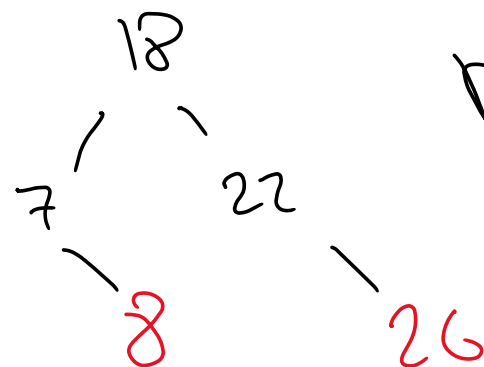
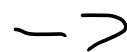
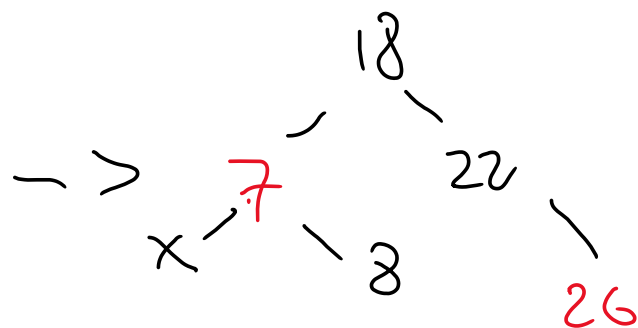
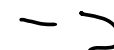
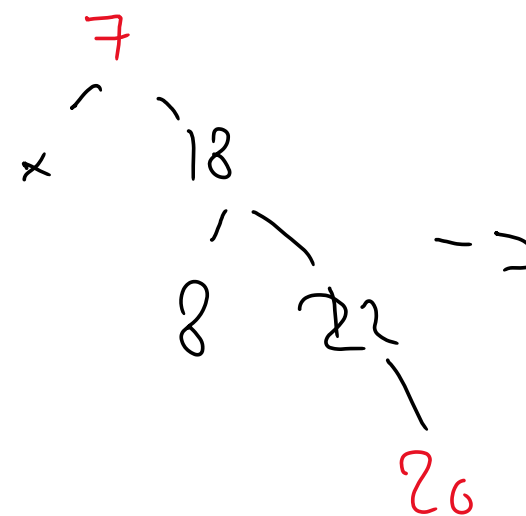
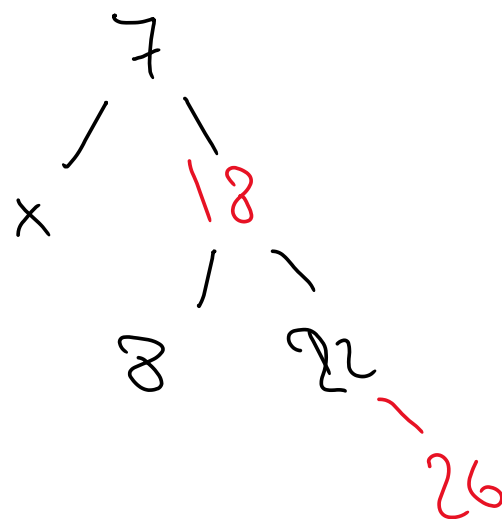
pd(10)



Del(10)



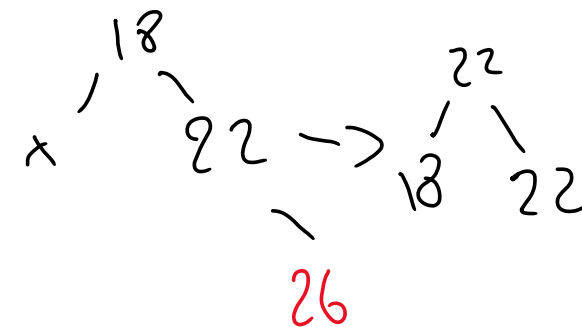
Del(3)



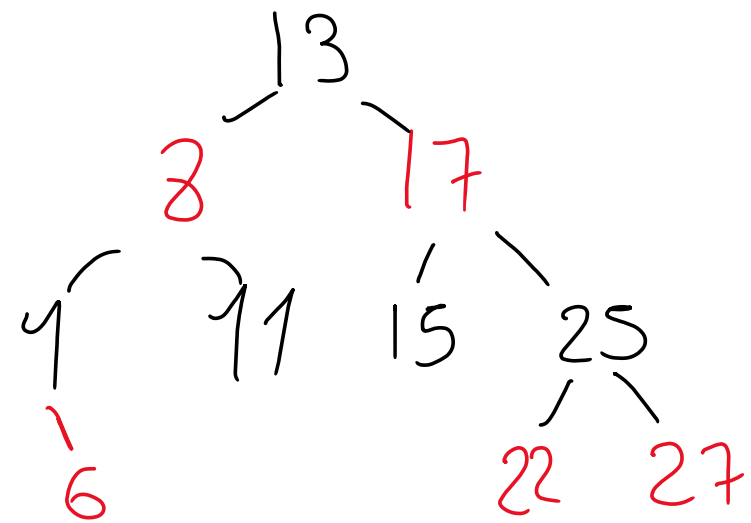
Del(7)



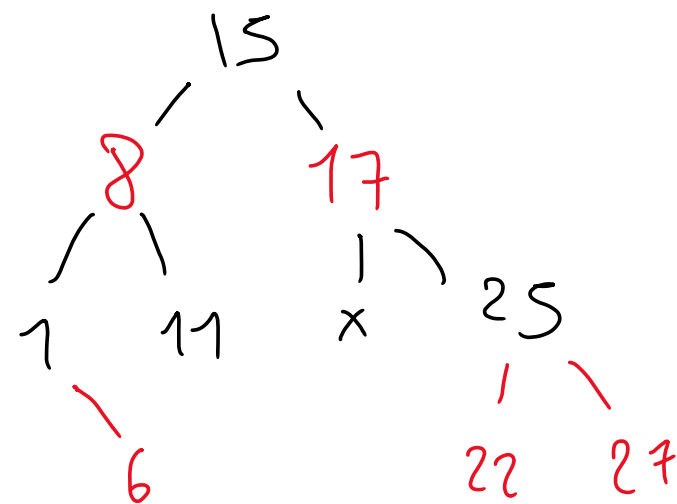
Del(8)



x = Double Black

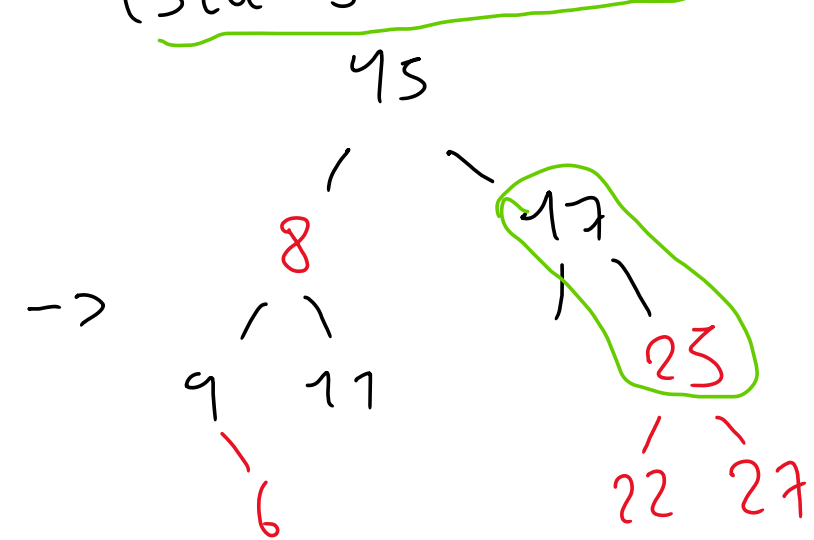


Del(13)  
→

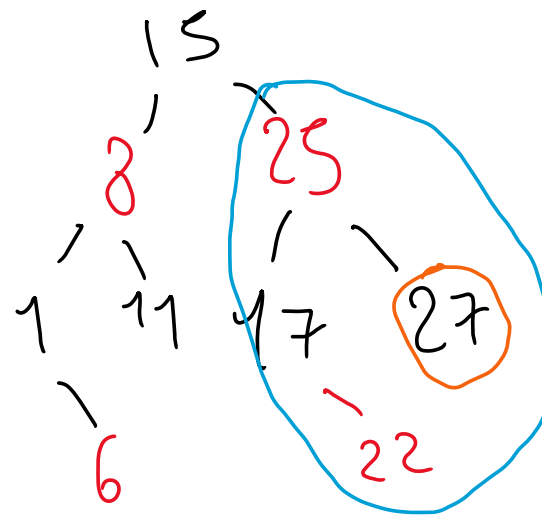


→

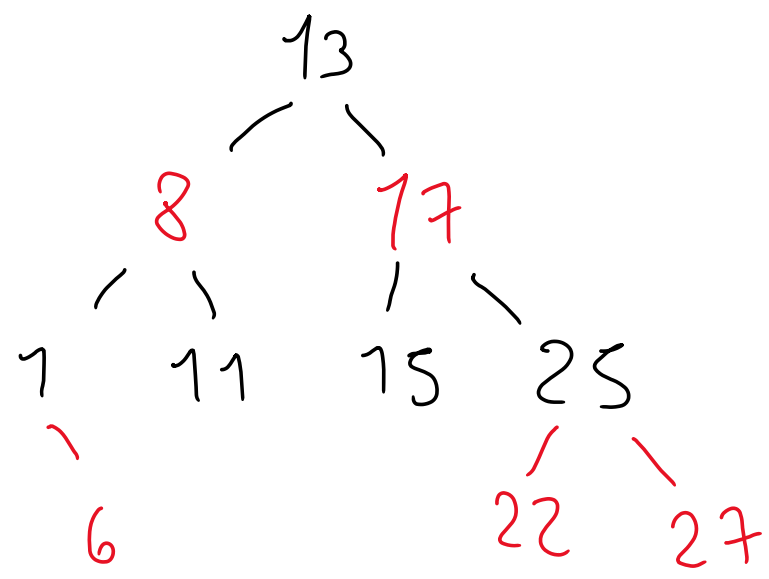
(scambio colore w e g e zoota g, rimovi: DB colore solo figlio o altro o: w o: nero)



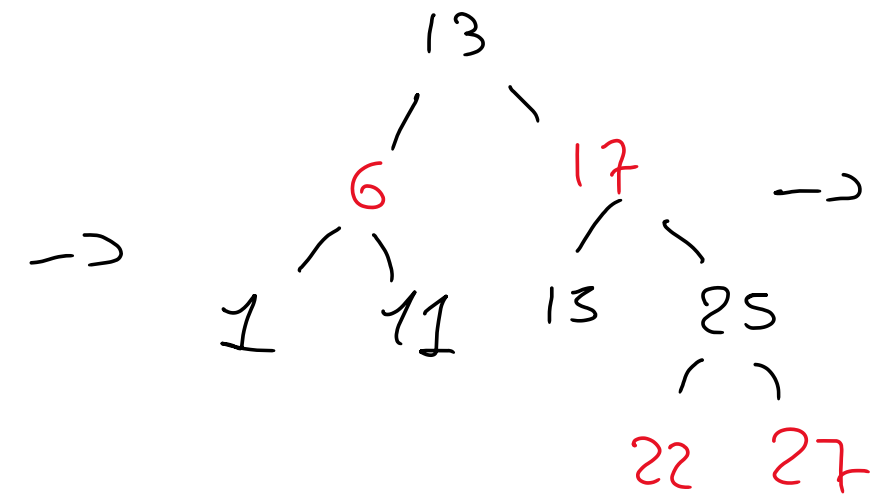
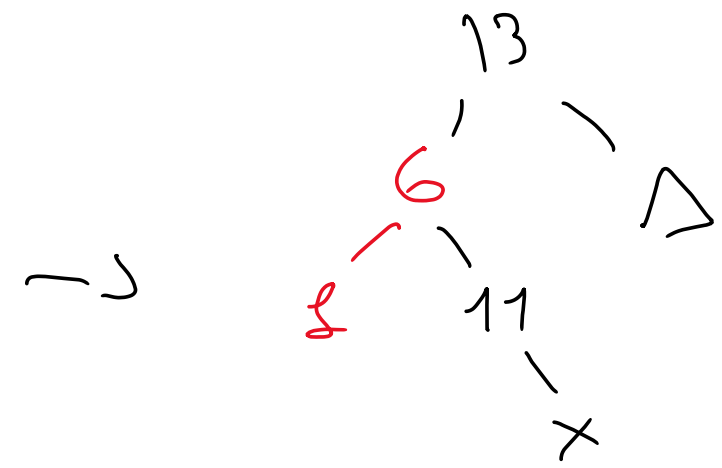
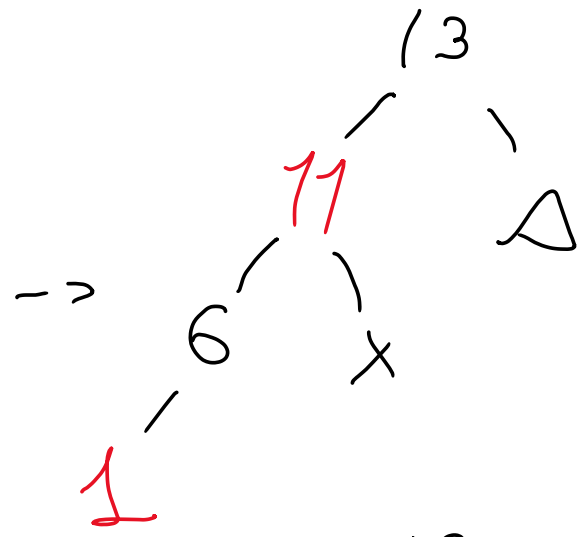
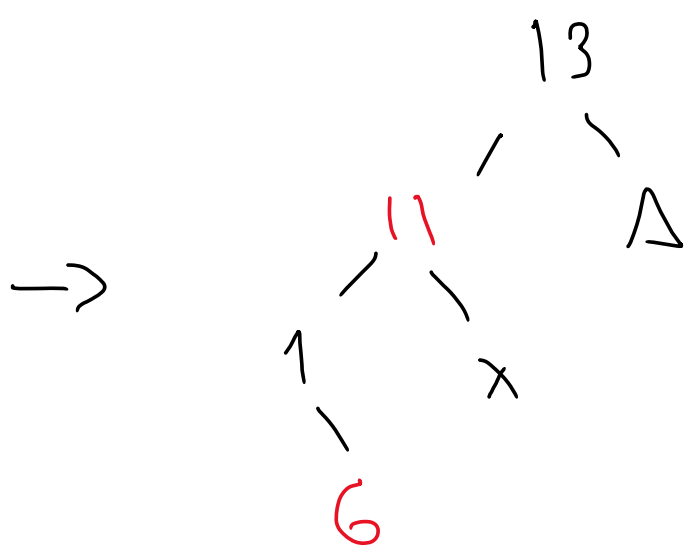
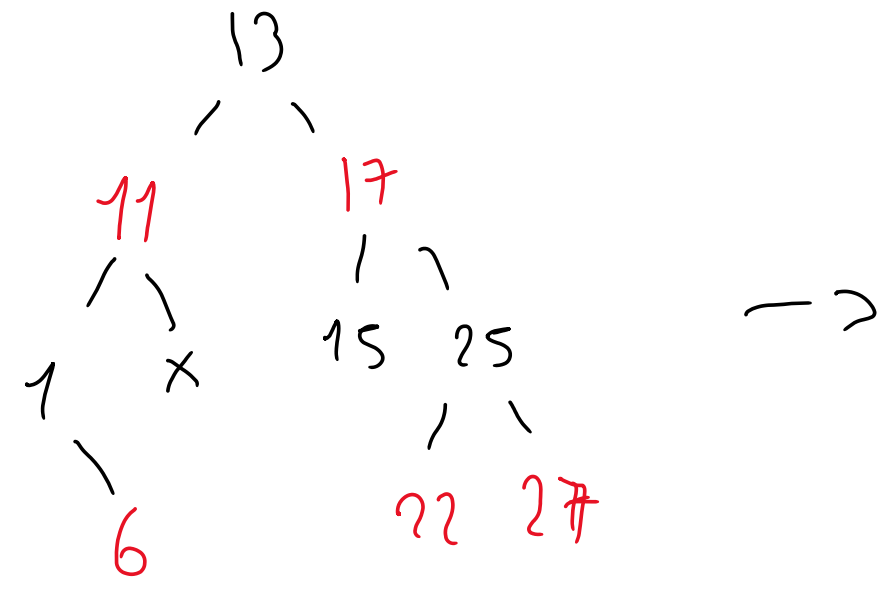
→



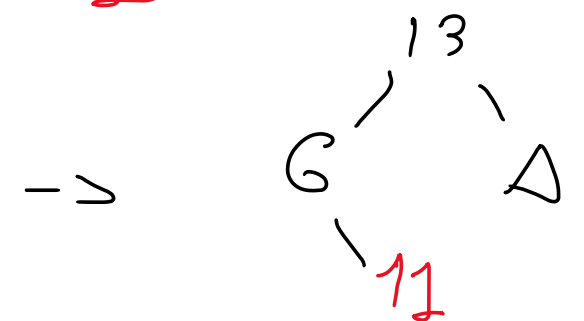
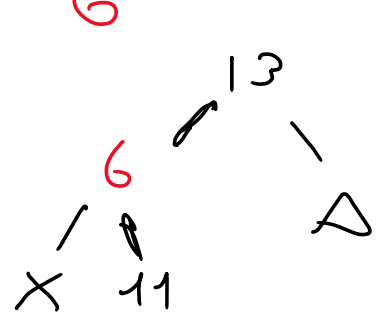
x: DB



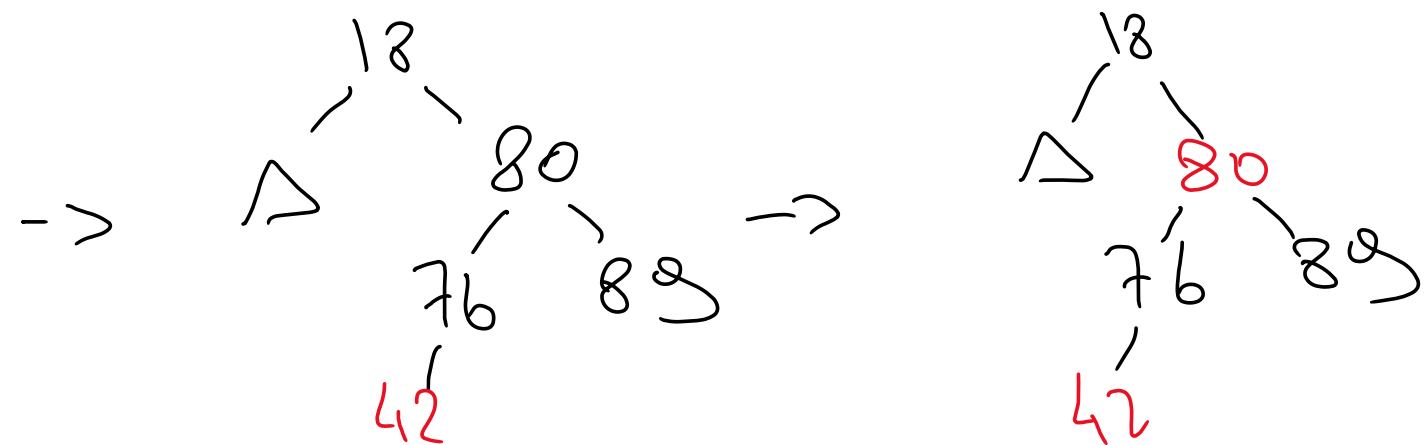
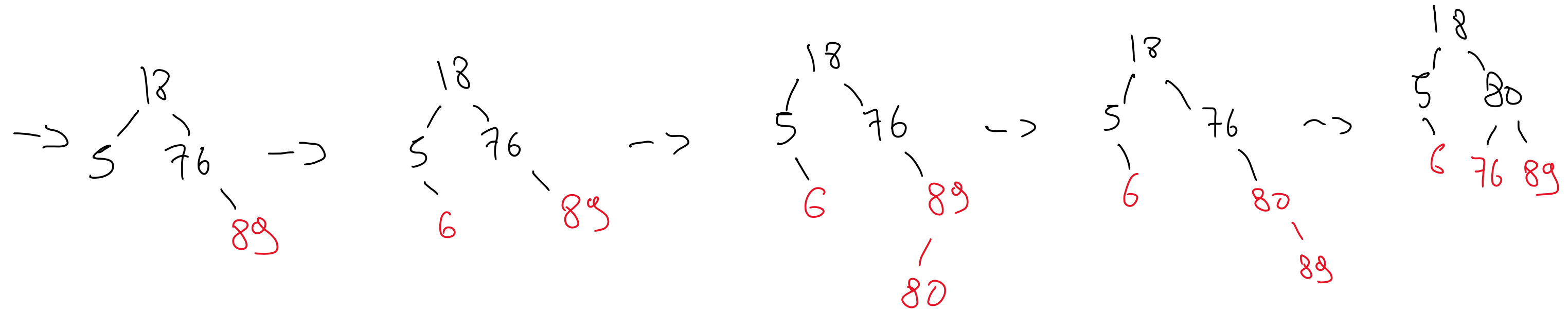
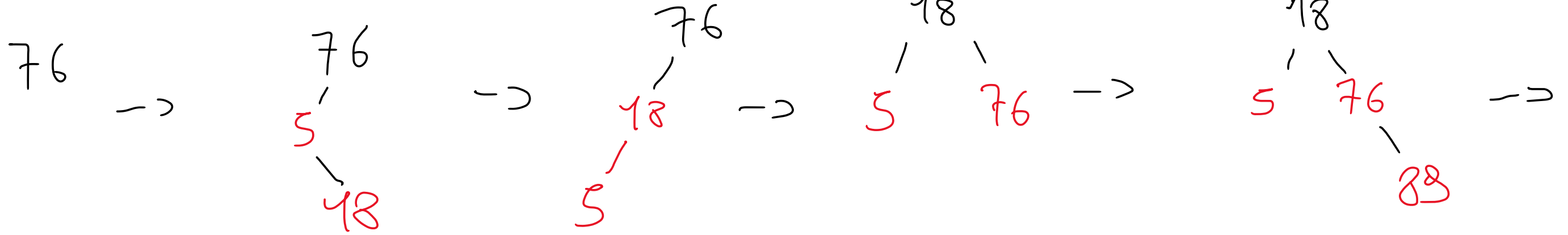
Del(8)  
→



Del(1)  
→

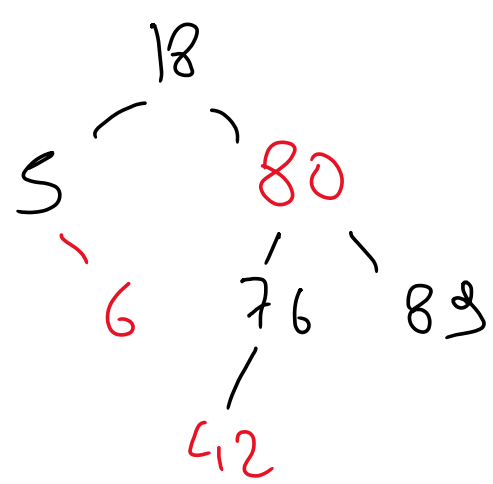


$\langle 76, 5, 18, 89, 6, 80, 42, 56, 12, 31 \rangle$

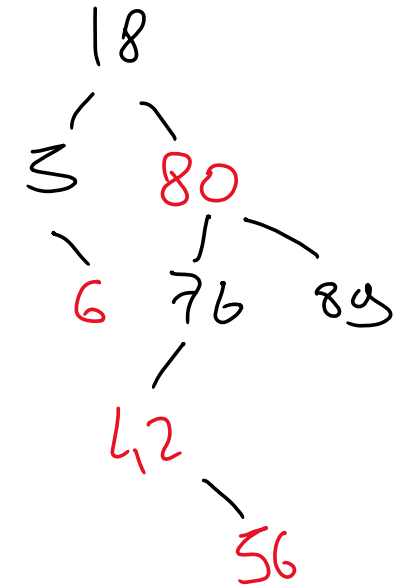




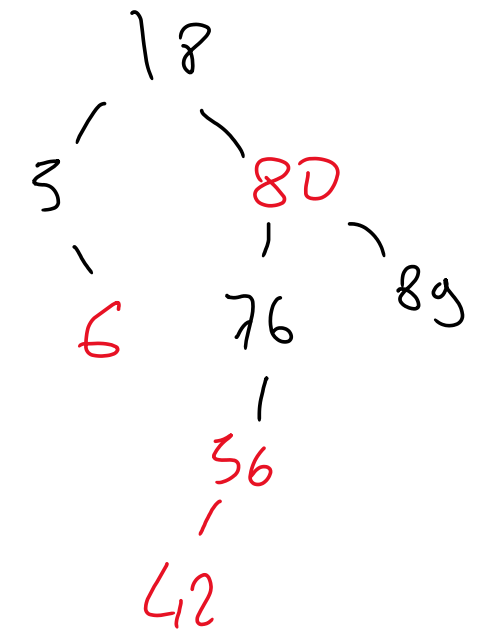
$\langle 76, 5, 18, 89, 6, 80, 42, 56, 12, 31 \rangle$



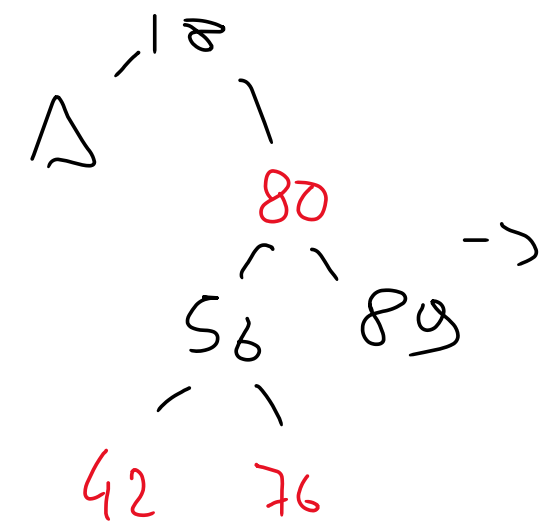
$\rightarrow$



$\rightarrow$



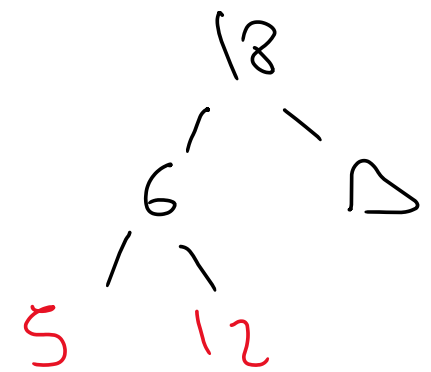
$\rightarrow$



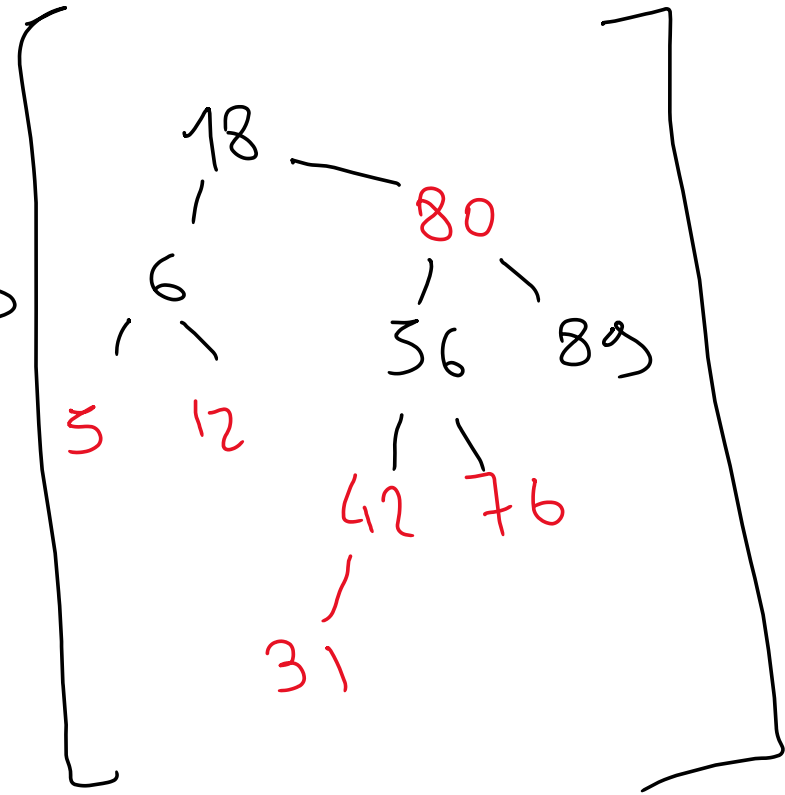
$\rightarrow$

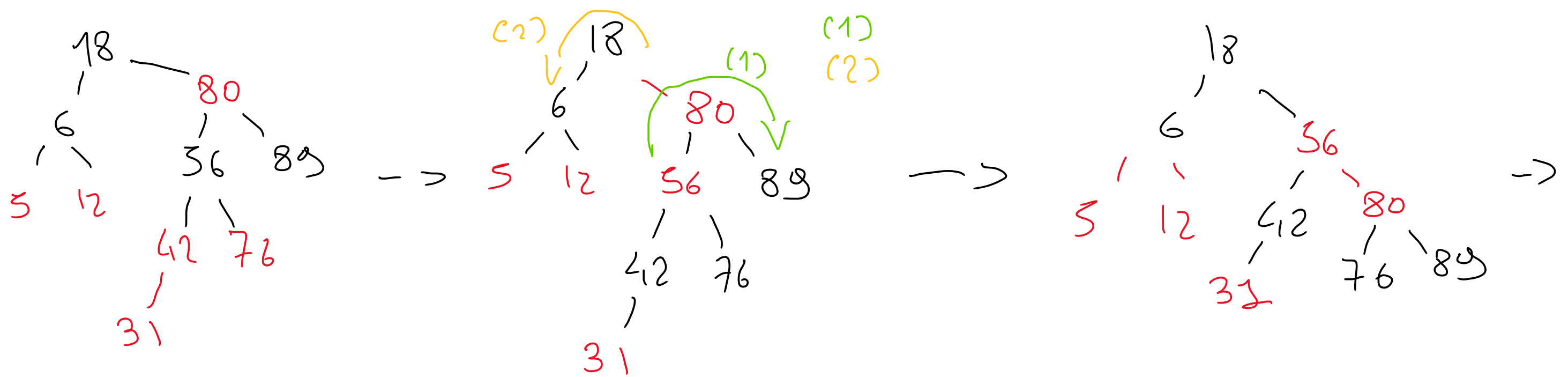


$\rightarrow$

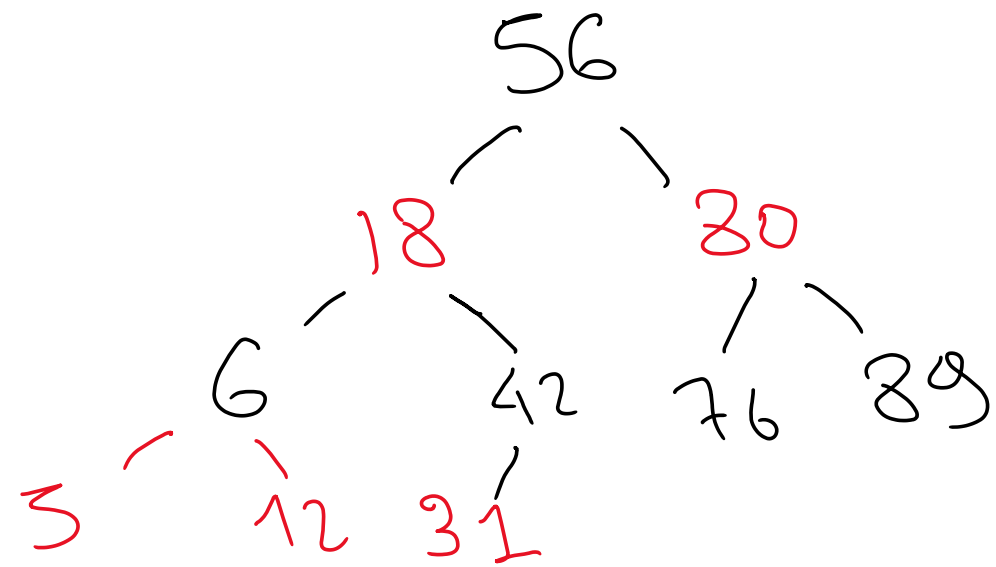


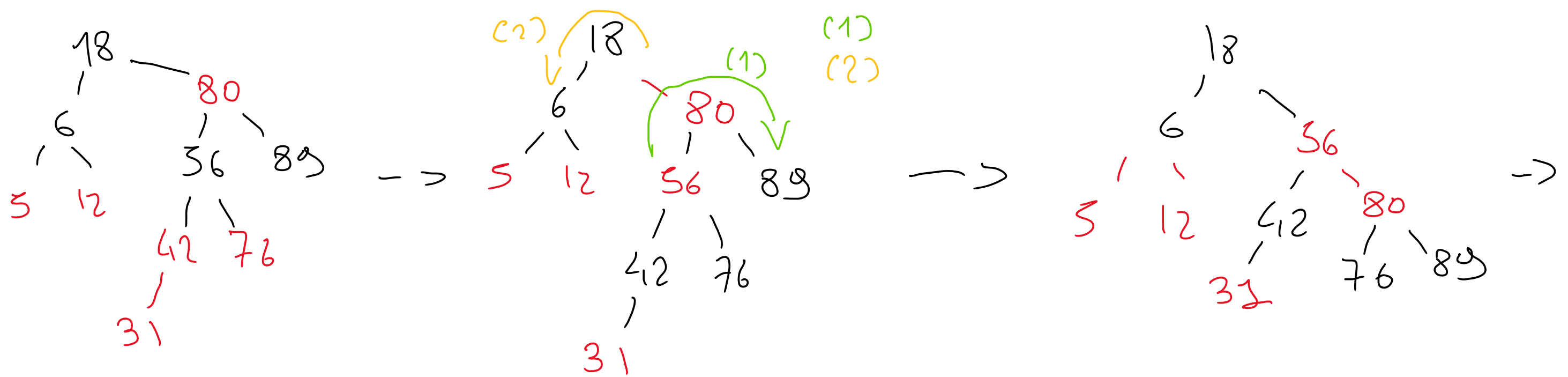
$\rightarrow$



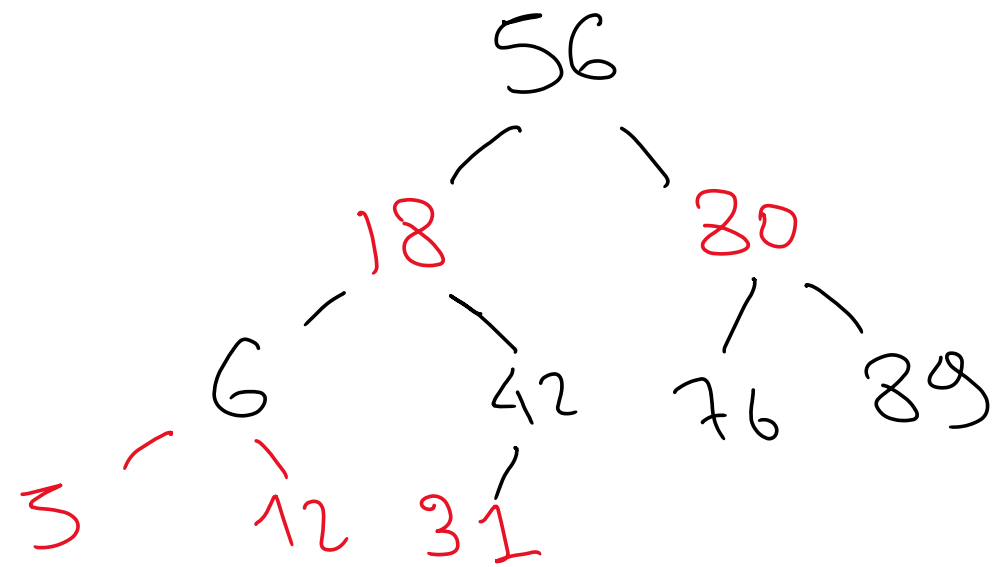


(insertion)



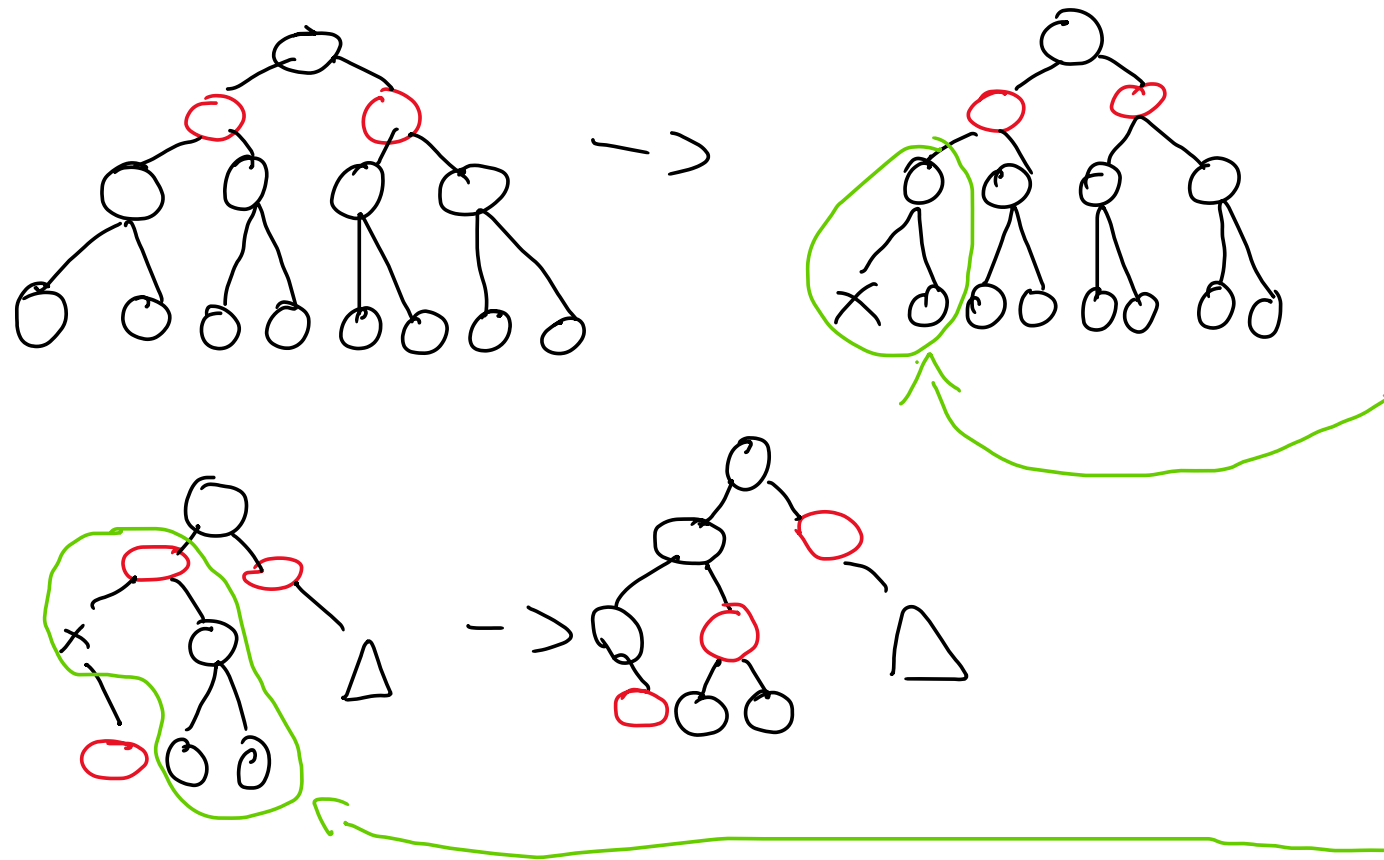


(insertion)



$x = DB$

2. Si supponga di operare su di un albero Rosso-Nero completo, contenente 15 chiavi. I nodi dell'albero sono tutti nodi neri ad esclusione dei nodi del livello 1 il cui colore è rosso. Nello specifico si effettuino 6 operazioni di cancellazioni della chiave più piccola contenuta nell'albero e si fornisca la configurazione dell'albero dopo ciascuna delle 6 cancellazioni.



W nero, entrambi figli neri,  
y (P[x]) nero

1. x diventa nero (perde il doppio nero)
2. W diventa rosso
3. P[x] diventa doppio nero
4. Ricomincia considerando p[x] come nuova x

W nero, entrambi figli neri,  
y (P[x]) rosso

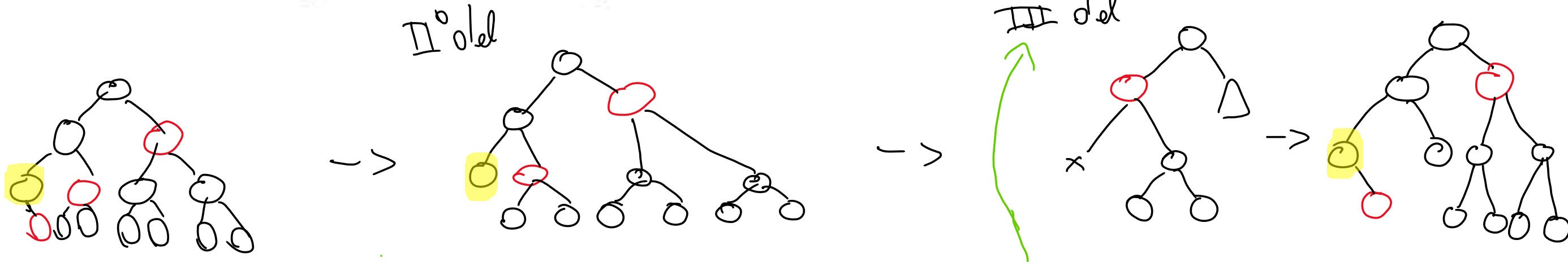
1. x diventa nero
2. W diventa rosso
3. P[x] diventa nero

$x^o del$

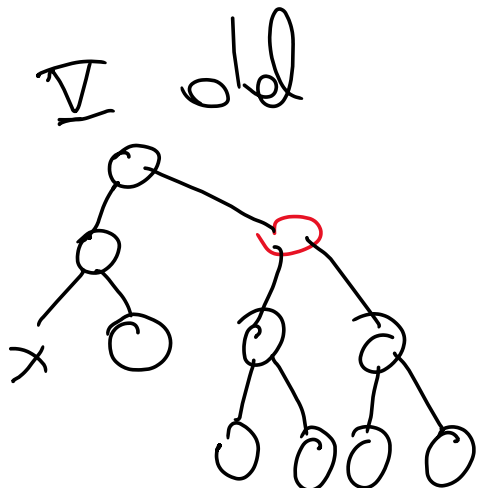
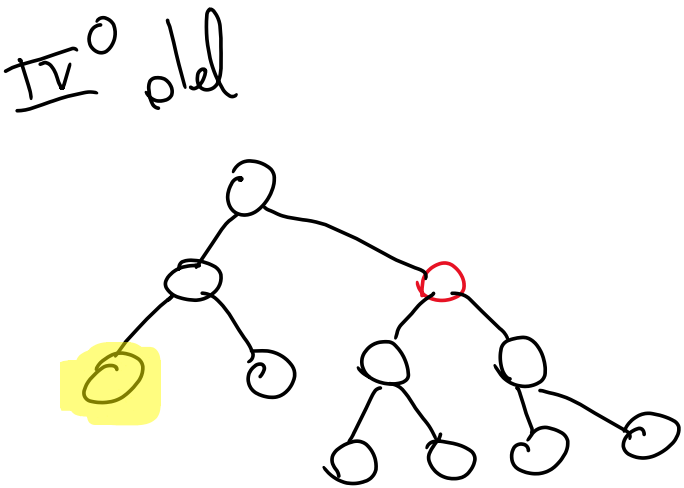
Esercizi da esame:

2. Si supponga di operare su di un albero Rosso-Nero completo, contenente 15 chiavi. I nodi dell'albero sono tutti nodi neri ad esclusione dei nodi del livello 1 il cui colore è rosso. Nello specifico si effettuino 6 operazioni di cancellazioni della chiave più piccola contenuta nell'albero e si fornisca la configurazione dell'albero dopo ciascuna delle 6 cancellazioni.

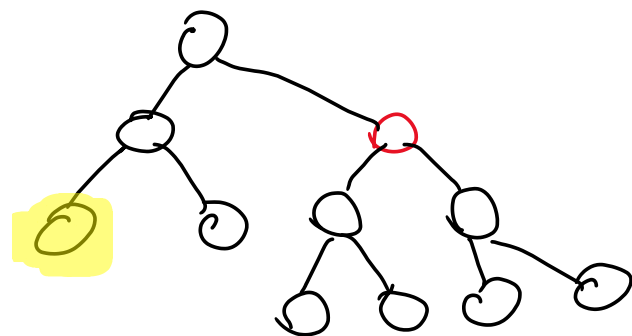
$x = DB$   
■ = nodo o/u cancellato



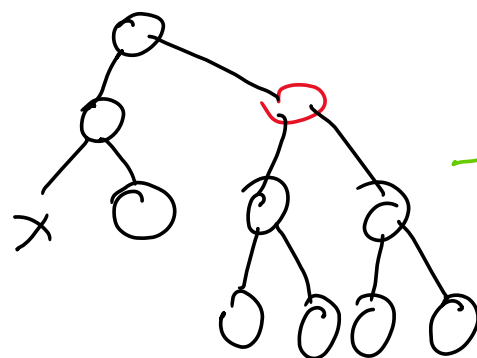
- W rosso
1. W diventa nero
  2. P[x] diventa rosso
  3. P[x] ruota a sinistra
  4. Siamo a uno dei casi precedenti



IV old

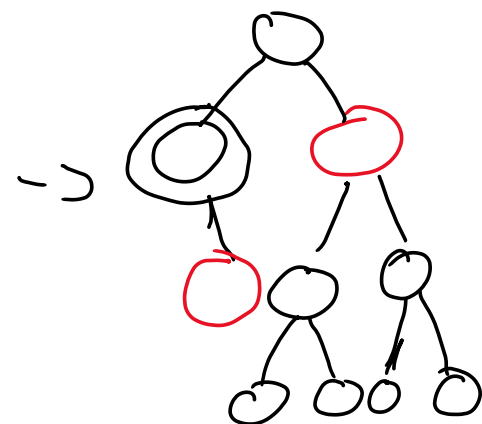
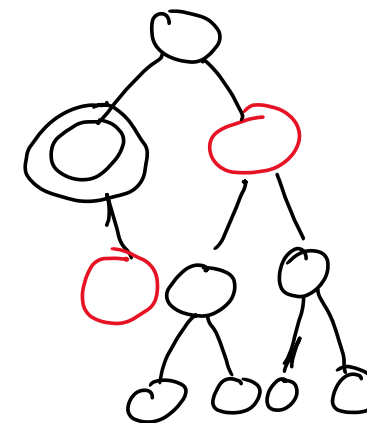


V old



W nero, entrambi figli neri,  
y (P[x]) nero

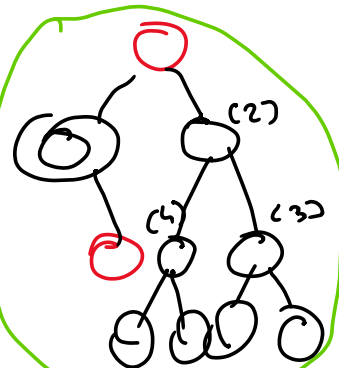
1. x diventa nero (perde il doppio nero)
2. W diventa rosso
3. P[x] diventa doppio nero
4. Ricomincia considerando p[x] come nuova x



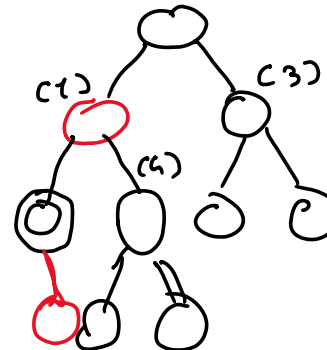
W rosso

1. W diventa nero
2. P[x] diventa rosso
3. P[x] ruota a sinistra
4. Siamo a uno dei casi precedenti

(1)

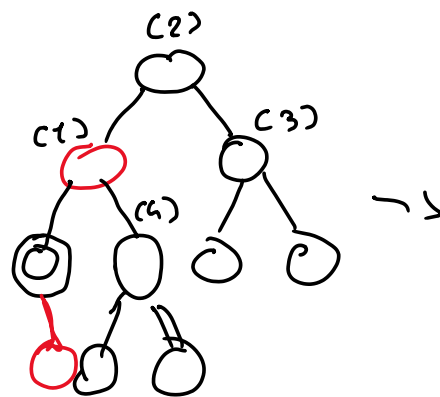
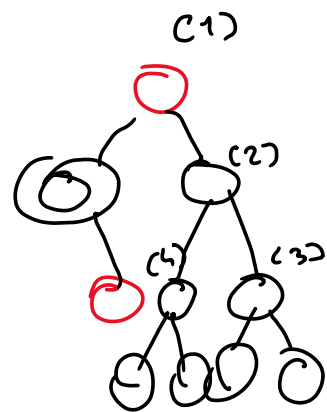


(2)



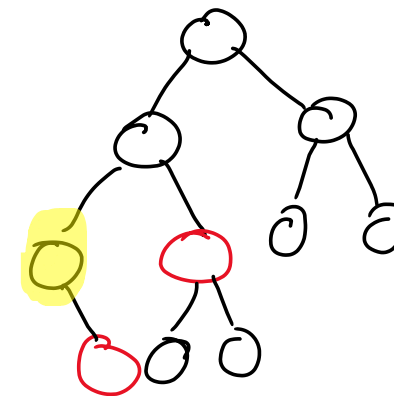
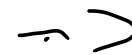
W nero, entrambi figli neri,  
y (P[x]) rosso

1. x diventa nero
2. W diventa rosso
3. P[x] diventa nero

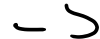
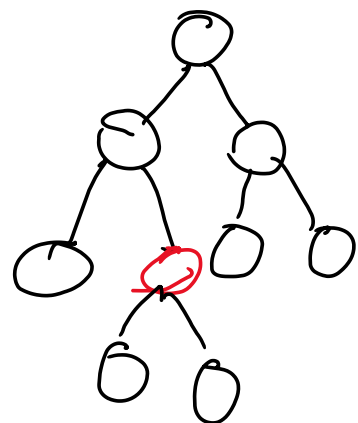


W nero, entrambi figli neri,  
y (P[x]) rosso

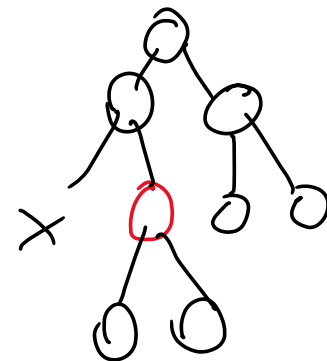
1. x diventa nero
2. W diventa rosso
3. P[x] diventa nero



VI del:

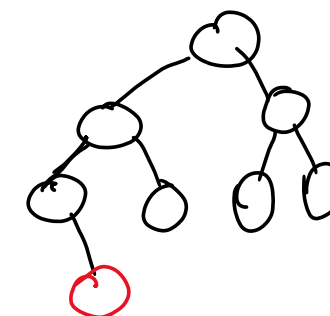
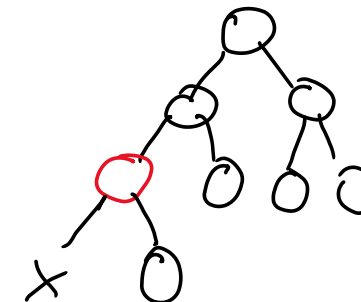
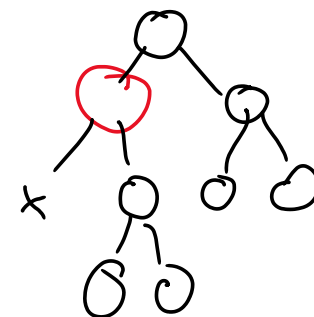


VII del:

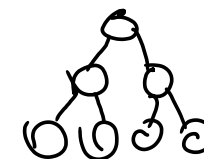


W rosso

1. W diventa nero
2. P[x] diventa rosso
3. P[x] ruota a sinistra
4. Siamo a uno dei casi precedenti



VIII del



W nero, entrambi figli neri,  
y (P[x]) rosso

1. x diventa nero
2. W diventa rosso
3. P[x] diventa nero